CLAUDE CERTIFIED ARCHITECT - FOUNDATIONS | EXAM CODE CCAR-F

Mock Examination 03

Claude Certified Architect - Foundations
60 scenario-based questions | 120 minutes

Scenarios assessed in this paper: Scenario 3 - Multi-Agent Research System; Scenario 4 - Developer Productivity with Claude; Scenario 5 - Claude Code for Continuous Integration; Scenario 6 - Structured Data Extraction

Format: multiple-choice and multiple-response; every question states how many answers to select.

Contents: instructions and blueprint | candidate answer sheet | 60 questions | quick answer key and score tracker | detailed explanations for every option | domain-level review guide | revision checklist.

Status: ORIGINAL PRACTICE MATERIAL. This is not an official Anthropic examination paper and contains no confidential exam content. It was written against the publicly available Claude Certified Architect - Foundations Exam Guide (v1.0, effective July 2026).

Contents

1. Instructions and exam conditions 3

2. What is verified, what is supplied, and what is original 3

3. Exam blueprint for this paper 4

Candidate answer sheet 5

4. Examination - Questions 1 to 60 6

Scenario 3: Multi-Agent Research System (Questions 1-15) 6

Scenario 4: Developer Productivity with Claude (Questions 16-32) 10

Scenario 5: Claude Code for Continuous Integration (Questions 33-47) 14

Scenario 6: Structured Data Extraction (Questions 48-60) 17

5. Quick answer key and score tracker 21

6. Detailed answer key and explanations 22

7. Domain-level performance review guide 46

Domain 1 - Agentic Architecture & Orchestration (27% official weight; 16 questions here) 46

Domain 2 - Tool Design & MCP Integration (18% official weight; 11 questions here) 46

Domain 3 - Claude Code Configuration & Workflows (20% official weight; 12 questions here) 46

Domain 4 - Prompt Engineering & Structured Output (20% official weight; 12 questions here) 46

Domain 5 - Context Management & Reliability (15% official weight; 9 questions here) 47

8. Final revision checklist 48

1. Instructions and exam conditions

Time limit: 120 minutes for 60 questions (an average of 2 minutes per question). Work in one sitting, without notes, documentation, or AI assistance, to get a realistic read on your readiness.

• Every question is anchored to one of the production scenarios in this paper. Read the scenario description at the start of each scenario section; questions are otherwise self-contained.
• Each question states **how many answers to select**. For 'Select ONE', exactly one option is best. For 'Select exactly TWO/THREE', all selected options must be correct; there is no partial credit in this practice format.
• Choose the *best* answer for the stated constraints, not an answer that could merely work. Where a question asks for the FIRST, SAFEST, or MOST RELIABLE step, that qualifier is part of the question.
• Attempt every question. Unanswered questions score zero. There is no 'all of the above' or 'none of the above'.
• Do not look at the answer key until you have finished. The key, explanations, and review guide begin after Question 60.

2. What is verified, what is supplied, and what is original

Category	What it covers
Verified official information	From the official Exam Guide (CCAR-F v1.0): 60 items; multiple-choice and multiple-response items that state how many responses to select; 4 scenarios drawn from a bank of 6; 120-minute limit; scaled passing score of 720 on a 100-1,000 scale; score report with percent-correct by domain; domain weights 27/18/20/20/15%; the 31 task statements and the in-scope/out-of-scope lists.
Scenario descriptions	The six production scenarios are quoted from the official guide; the extra topic lists you were given (e.g., idempotency, structured handoffs, schema design) guided question themes. Scenario text shown above each question group is the official description.
Original practice content	All questions, options, explanations, and review guidance in this file were written for this pack. They are not official questions, are not copied from commercial practice exams or leaked banks, and may differ in difficulty from the real exam.
Supplemental knowledge	A few items use general engineering knowledge to make a scenario realistic (for example CI/CD terminology or least-privilege shell access). They are flagged 'Supplemental' in the explanation and are not confirmed exam objectives.

A raw percentage on this paper is **not** a scaled score. The real exam reports a scaled score (100-1,000) with a cut score of 720 derived from a formal standard-setting study; percent-correct by domain is shown for information only. Doing well on these papers does not guarantee a pass.

3. Exam blueprint for this paper

Domain	Official weight	Questions in this paper
1. Agentic Architecture & Orchestration	27%	16
2. Tool Design & MCP Integration	18%	11
3. Claude Code Configuration & Workflows	20%	12
4. Prompt Engineering & Structured Output	20%	12
5. Context Management & Reliability	15%	9
Total	**100%**	**60**

Question counts are the closest integer allocation of the official weights to 60 questions. Each question has exactly one primary domain; secondary domains appear in the answer key where a question also touches another domain.

Scenario	Questions	Numbers
Scenario 3 - Multi-Agent Research System	15	1-15
Scenario 4 - Developer Productivity with Claude	17	16-32
Scenario 5 - Claude Code for Continuous Integration	15	33-47
Scenario 6 - Structured Data Extraction	13	48-60

Design targets (not claims about the real exam): about 20% foundational, 55% intermediate, 25% challenging. In this paper: 12 foundational, 33 intermediate, 15 challenging. Response types: 54 single-answer, 5 select-two, 1 select-three.

Candidate answer sheet

Mark your answers here while taking the paper. Cells that do not exist for a question are greyed out. Where a question says 'Select exactly TWO/THREE', mark exactly that many letters. Do not turn to the answer key until you have finished all 60 questions (or the 120 minutes have elapsed).

Name: _____ Date: _____ Start time: _____ End time: _____

No.	Pick	A	B	C	D	E	F
1	1	(A)	(B)	(C)	(D)		
2	1	(A)	(B)	(C)	(D)		
3	1	(A)	(B)	(C)	(D)		
4	1	(A)	(B)	(C)	(D)		
5	1	(A)	(B)	(C)	(D)		
6	1	(A)	(B)	(C)	(D)		
7	1	(A)	(B)	(C)	(D)		
8	2	(A)	(B)	(C)	(D)	(E)	
9	1	(A)	(B)	(C)	(D)		
10	1	(A)	(B)	(C)	(D)		
11	1	(A)	(B)	(C)	(D)		
12	1	(A)	(B)	(C)	(D)		
13	2	(A)	(B)	(C)	(D)	(E)	
14	3	(A)	(B)	(C)	(D)	(E)	(F)
15	1	(A)	(B)	(C)	(D)		
16	1	(A)	(B)	(C)	(D)		
17	1	(A)	(B)	(C)	(D)		
18	1	(A)	(B)	(C)	(D)		
19	1	(A)	(B)	(C)	(D)		
20	1	(A)	(B)	(C)	(D)		
21	1	(A)	(B)	(C)	(D)		
22	1	(A)	(B)	(C)	(D)		
23	1	(A)	(B)	(C)	(D)		
24	1	(A)	(B)	(C)	(D)		
25	1	(A)	(B)	(C)	(D)		
26	1	(A)	(B)	(C)	(D)		
27	1	(A)	(B)	(C)	(D)		
28	1	(A)	(B)	(C)	(D)		
29	1	(A)	(B)	(C)	(D)		
30	1	(A)	(B)	(C)	(D)		

No.	Pick	A	B	C	D	E	F
31	1	(A)	(B)	(C)	(D)		
32	1	(A)	(B)	(C)	(D)		
33	1	(A)	(B)	(C)	(D)		
34	1	(A)	(B)	(C)	(D)		
35	2	(A)	(B)	(C)	(D)	(E)	
36	1	(A)	(B)	(C)	(D)		
37	1	(A)	(B)	(C)	(D)		
38	1	(A)	(B)	(C)	(D)		
39	1	(A)	(B)	(C)	(D)		
40	1	(A)	(B)	(C)	(D)		
41	1	(A)	(B)	(C)	(D)		
42	1	(A)	(B)	(C)	(D)		
43	1	(A)	(B)	(C)	(D)		
44	1	(A)	(B)	(C)	(D)		
45	1	(A)	(B)	(C)	(D)		
46	1	(A)	(B)	(C)	(D)		
47	1	(A)	(B)	(C)	(D)		
48	1	(A)	(B)	(C)	(D)		
49	1	(A)	(B)	(C)	(D)		
50	1	(A)	(B)	(C)	(D)		
51	1	(A)	(B)	(C)	(D)		
52	1	(A)	(B)	(C)	(D)		
53	1	(A)	(B)	(C)	(D)		
54	2	(A)	(B)	(C)	(D)	(E)	
55	1	(A)	(B)	(C)	(D)		
56	1	(A)	(B)	(C)	(D)		
57	2	(A)	(B)	(C)	(D)	(E)	
58	1	(A)	(B)	(C)	(D)		
59	1	(A)	(B)	(C)	(D)		
60	1	(A)	(B)	(C)	(D)		

4. Examination - Questions 1 to 60

Time allowed: 120 minutes. Start the clock now.

Scenario 3: Multi-Agent Research System (Questions 1-15)

You are building a multi-agent research system using the Claude Agent SDK. A coordinator agent delegates to specialized subagents: one searches the web, one analyzes documents, one synthesizes findings, and one generates reports. The system researches topics and produces comprehensive, cited reports.

Question 1

Select ONE answer.

A report-generation subagent runs an agentic loop that treats any response whose stop_reason is not 'tool_use' as finished and returns the text. Users occasionally receive reports that end mid-sentence in the third section. Logs show those responses ended with stop_reason 'max_tokens'. What is the right change to the loop?

A. Handle stop_reason explicitly: treat 'end_turn' as complete, and for 'max_tokens' do not ship the output as final but continue generation or retry with a higher output limit.

B. Keep treating every non-tool_use response as final, but add a check that the text ends with a period or other closing punctuation before the report is returned to the user, and retry once if it does not.

C. Raise the iteration cap so the loop gets more chances to finish the report before it stops.

D. Instruct the model in the system prompt to keep each report short enough that it never reaches the output limit.

Question 2

Select ONE answer.

Three research subagents (web, papers, news) each return findings, and the final report contains the same statistic cited three times from three outlets that all quote one original study. Where should deduplication of overlapping sources be handled?

A. In the report-generation subagent, which should detect duplicates while formatting the final document.

B. In the coordinator, which sees every subagent's results and is responsible for aggregation and reconciling overlap before synthesis.

C. In each subagent, by instructing each one to check whether either of the other two might already have found the same source and to skip it if so.

D. In the user's review step, by asking readers to flag repeated citations after the report is delivered.

Question 3

Select ONE answer.

A developer defines the web research subagent with a one-word description ('Researcher') and gives it every tool the coordinator has, including file writes. The coordinator sometimes delegates document analysis to it by mistake, and it has overwritten a working file. What configuration is best?

A. Define the subagent with a specific description of its role, a focused system prompt, and a restricted tool list limited to search and fetch tools.

B. Rename the subagent to 'WebResearcher' and leave the description and tool list unchanged.

C. Keep the broad tool list but add a line to the coordinator's prompt telling it which tasks should never be sent to the web researcher.

D. Remove the subagent and have the coordinator perform web research itself so there is no delegation to get wrong.

Question 4

Select ONE answer.

The document analysis subagent returns summaries as free-form paragraphs. The synthesis subagent then merges them and the final report cannot say which page or document any claim came from. Which handoff format preserves attribution?

A. Structured records that separate each claim's content from its metadata: source document, page number, URL, and publication date.

B. Short summaries without any metadata so the synthesis agent can focus on content.

C. A single combined paragraph with all claims followed by a bibliography of every document consulted.

D. Longer free-form paragraphs where the analyst mentions the source document name and page at the start of each paragraph, so the writer can see them.

Question 5

Select ONE answer.

A team hard-coded the research agent's workflow: always run three web searches, then one document analysis, then synthesis. Many queries need only one search, while some need follow-up searches triggered by what the first results revealed. What approach is better?

A. Keep the fixed sequence but increase it to five searches, so that complex queries are covered as well, and skip the fifth when the first four agree.

B. Let Claude decide which tool to call next from the context and results so far, within the agentic loop, with guardrails for cost and safety.

C. Run a classifier model to predict how many searches each query will need and schedule exactly that many.

D. Add a decision tree in code that checks keywords in the query and selects among ten pre-defined sequences.

Question 6

Select ONE answer.

For the topic 'adoption of renewable energy in emerging markets', the coordinator creates one subtask: 'solar power in India'. Every subagent performs well, but the final report ignores wind, hydro, other regions, and policy. What is the best fix?

A. Run the same single subtask five times with different search engines to gather more material.

B. Instruct the web search subagent to broaden its queries beyond the assigned subtask whenever it feels a topic is incomplete, and to report on anything else that it finds along the way.

C. Add more synthesis instructions telling the synthesis subagent to highlight everything the report is missing.

D. Improve the coordinator's decomposition by prompting it to enumerate the topic's dimensions (technologies, regions, policy, finance) and add a coverage review that re-delegates gaps.

Question 7

Select ONE answer.

The research system exposes three capabilities through one MCP server: (1) a listing of the document collections and their coverage dates, (2) running a keyword query against a collection, and (3) fetching a document by ID. Which design best follows MCP conventions?

A. Put the collection catalog in the system prompt and remove the listing capability from the server.

B. Expose all three as resources so the agent can read them whenever it needs.

C. Expose the collection catalog as an MCP resource and provide the query and fetch operations as tools.

D. Expose all three as tools and have the agent call the listing tool before every query.

Question 8

Select exactly TWO answers (choose 2).

You are assigning tools to research subagents. Select the TWO designs that best follow good tool-distribution practice.

A. Give the synthesis subagent unrestricted web search so it never needs to wait for the coordinator.

B. Give the synthesis subagent a narrow verify_fact tool for simple lookups and route complex verification through the coordinator.

C. Give every subagent the full tool set so that any of them can complete any task without delegation.

D. Remove all tools from subagents and have the coordinator execute every tool call on their behalf.

E. Give the web search subagent only search and load-document tools and keep file-writing tools with the report generator.

Question 9

Select ONE answer.

The document analysis subagent has one tool, analyze_document. Depending on the call it returns a summary, a list of data points, or a verdict on a claim, so downstream agents cannot rely on the output shape. What is the best redesign?

A. Keep one tool and add a free-text 'output_mode' parameter in which the model writes whatever format it wants, so that each caller can ask for the shape it needs from the document at that moment.

B. Keep one tool and ask downstream agents to detect the output shape and adapt their parsing.

C. Split it into purpose-specific tools such as extract_data_points, summarize_content, and verify_claim_against_source, each with a defined input and output contract.

D. Remove the tool and let the subagent answer from its own knowledge to avoid format problems.

Question 10

Select ONE answer.

A search tool rejects a query because it exceeds the 500-character limit. The tool returns an error flagged as isError with category 'validation'. How should the agent respond?

A. Report to the user that the search service is down and stop researching this topic.

B. Shorten or rephrase the query to meet the limit and call the tool again, since the error is caused by the input rather than a temporary outage.

C. Retry the identical call after a short delay with exponential backoff, as most tool errors clear up on their own once the service has recovered.

D. Escalate to a human reviewer as a permission problem and wait for access to be granted.

Question 11

Select ONE answer.

The document-analysis subagent can call a generic fetch_url tool. In testing, it sometimes fetches arbitrary pages unrelated to research, including internal admin URLs. What is the best way to reduce this risk while keeping the capability it needs?

A. Keep fetch_url and tell the subagent in its prompt to fetch only research documents and avoid anything internal.

B. Replace fetch_url with a load_document tool that validates the URL against allowed document sources and types before fetching.

C. Give the subagent access to the coordinator's web search tools instead of fetch_url so it can find documents itself.

D. Log every fetched URL and review the logs weekly for anything suspicious.

Question 12

Select ONE answer.

A verification subagent reviews each claim in a draft report and outputs a verdict. Editors can only manually check about 30 claims per report, and the reports contain about 200. How can the verification pass help them prioritize?

A. Send editors a random sample of 30 claims from each report, and assume that the remaining claims follow the same quality as the ones that were sampled, adjusting the sample size over time.

B. Ask the verifier to rate its overall confidence for the whole report and review the report only if the rating is below 5.

C. Have the verifier output a confidence value with each finding, calibrate thresholds on a labeled validation set, and route low-confidence ones to editors.

D. Treat every claim the verifier marks as supported as final and send all unsupported claims to editors, regardless of count.

Question 13

Select exactly TWO answers (choose 2).

Two credible sources report different figures for 2023 solar capacity added: one says 340 GW, another says 410 GW. Looking closer, one was published in January 2024 and the other in September 2024. Select the TWO practices that best handle this in the report.

A. Choose whichever source is cited most often in the other documents.

B. Report the average of the two figures as the best estimate.

C. Present both values with their sources and an explanation of the conflict instead of silently choosing one.

D. Require publication or collection dates in subagent outputs so differences can be interpreted as timing rather than contradiction.

E. Silently use the most recent figure and omit the older one to keep the report simple.

Question 14

Select exactly THREE answers (choose 3).

A document-analysis subagent hits a transient timeout while processing a corpus. Select the THREE practices that best support recovery in the multi-agent system.

A. Return structured error context to the coordinator: failure type, attempted query, partial results, and alternatives.

B. Return a generic status such as 'analysis unavailable' so the coordinator is not confused by details.

C. Terminate the whole research workflow on the first error to avoid incomplete reports.

D. Annotate the final synthesis with coverage information stating which topics are well supported and which have gaps.

E. Retry locally for the transient failure and propagate only if it cannot be resolved.

F. Return an empty analysis marked successful so the pipeline continues smoothly.

Question 15

Select ONE answer.

A document-analysis subagent processes 10 PDFs. Two are corrupted and cannot be parsed. What should the subagent return to the coordinator?

A. Results for the 8 readable documents, plus a structured error for each of the 2 failures stating what was attempted and why it failed.

B. Results for the 8 readable documents only, with no mention of the others so the output stays clean.

C. A single failure for the whole job with no partial results, because a partial analysis could mislead the coordinator about what the corpus contains.

D. Empty analyses for the 2 corrupted documents marked as successful, to keep the output format uniform.

Scenario 4: Developer Productivity with Claude (Questions 16-32)

You are building developer productivity tools using the Claude Agent SDK. The agent helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate code, and automate repetitive tasks. It uses the built-in tools (Read, Write, Bash, Grep, Glob) and integrates with Model Context Protocol (MCP) servers.

Question 16

Select ONE answer.

A coordinator for a codebase-understanding tool spawns three explorer subagents with the same instruction, 'Map this repository and report back'. They run in separate turns, return largely overlapping summaries, and none knows what the others found. Which redesign is best?

A. Keep three identical explorers but have the coordinator discard the two shortest summaries.

B. Reduce to one explorer and have it map the whole repository sequentially to avoid any overlap.

C. Give each explorer a distinct set of modules, include a shared baseline summary in each prompt, and spawn them together with multiple Task calls in one response.

D. Keep the same instruction for all three, but ask each explorer to read the others' output files as they finish and to skip any area already covered by someone else.

Question 17

Select ONE answer.

A team lists four tasks and asks which justify a coordinator with multiple subagents: (1) rename a method used in one file, (2) explain a single function's behavior, (3) survey five independent services to find how each handles retries, then compare them, (4) make a tightly coupled edit across three lines of one class. Which task best justifies multiple agents?

A. Task 3, because the services are independent, the exploration is verbose, and isolating each lets the coordinator compare compact results.

B. Task 1, because delegating even a small edit to a subagent isolates it from the main context and keeps that context clean.

C. Task 2, because explanation is easier for a specialized explainer agent than for the main agent.

D. Task 4, because tightly coupled edits benefit from splitting the lines among several agents.

Question 18

Select ONE answer.

Yesterday a developer explored a legacy module in a named session. Overnight, a refactor merged that renamed or moved about 60% of the files that session had read. Today they ask the resumed session to continue the analysis, and it cites file paths that no longer exist. What is the best approach?

A. Resume the session and tell it only that 'some files changed', then continue using its earlier findings as they are.

B. Start a new session seeded with a structured summary of the conclusions that still hold, and re-explore the parts that changed.

C. Fork the session so the stale analysis becomes the baseline of a new branch and continue from there.

D. Delete the notes and ask the agent to answer from general knowledge of the module's purpose.

Question 19

Select ONE answer.

An explorer subagent's report on a payment service covers charges and refunds but says nothing about webhooks, which the coordinator knows from the user's question are central. What should the coordinator do?

A. Have the explorer guess how webhooks typically work in payment systems and add a paragraph.

B. Ask the explorer to expand the whole report to be more comprehensive in general and resubmit it.

C. Evaluate the report against the goal, identify the webhook gap, and re-delegate a targeted exploration with a specific query before finalizing.

D. Accept the report and tell the user that webhooks were treated as out of scope for this run, offering to investigate them in a follow-up request.

Question 20

Select ONE answer.

You must change the default timeout from 30 to 60 in a single function inside a 2,000-line file. The text 'timeout = 30' appears exactly once. Which built-in tool is best suited?

A. Glob, with a pattern that matches 'timeout = 30' and replaces it.

B. Grep, with a replacement option to change the value in place.

C. Edit, replacing the unique text 'timeout = 30' with 'timeout = 60'.

D. Write, regenerating all 2,000 lines of the file with the new value.

Question 21

Select ONE answer.

A developer adds a personal MCP server to ~/.claude.json that exposes a tool find_code. The project's .mcp.json already provides search_code with a similar description. The agent now alternates unpredictably between them, and the developer wonders whether to disable the project server for their sessions. What is the correct understanding and fix?

A. The project server always takes priority over the user server, so the personal tool will never be chosen.

B. Add the personal server to the project's .mcp.json so that both tools are managed in one file.

C. Tools from all configured servers are available at once, so differentiate the two tool descriptions (or remove the personal tool) to eliminate the overlap.

D. Only the user-scoped server is active when both exist, so the alternating behavior must come from a model defect that should be reported to the Anthropic support team.

Question 22

Select ONE answer.

A developer wants to know which module prints the message 'Quota exceeded' and what calls it. Which sequence is most efficient using built-in tools?

A. Glob for files with 'quota' in their names, then Read all of them in full to find the message and check who calls the function.

B. Use Bash to concatenate every source file into one output and scan for the message.

C. Read the project's README and infer which module prints the message from its description.

D. Grep for the message to find the file, Read it to identify the function, then Grep for that function's name to find callers.

Question 23

Select ONE answer.

Your CLAUDE.md has grown to 900 lines covering testing, API conventions, and deployment. Several engineers edit it and collide on merges, and most sessions don't need all of it. What is the best restructuring?

A. Keep one file but move the least-used sections to the bottom, so that Claude is likely to ignore them and merge collisions become less frequent in the top half.

B. Move the whole file into each engineer's ~/.claude/CLAUDE.md so no one collides on merges.

C. Delete everything not related to testing, since tests are the main thing that must be consistent.

D. Split it into focused files under .claude/rules/ such as testing.md and deployment.md, scoping topic files with paths frontmatter where they apply to certain files, and keep only universal guidance in CLAUDE.md.

Question 24

Select ONE answer.

Engineers want a /design-options skill that brainstorms three alternative architectures for a feature. They find that the long brainstorming text clutters the main session and influences later implementation choices. How should the skill be configured?

A. Set context: fork in the skill's frontmatter so the brainstorming runs in an isolated context and only the outcome returns to the main session.

B. Put the skill in ~/.claude/skills/ so that each engineer's copy runs separately.

C. Add allowed-tools: Read so that the skill cannot write anything during the brainstorm.

D. Add the brainstorming prompt to CLAUDE.md so that it runs automatically at the start of every session and keeps the options in view for the whole team.

Question 25

Select ONE answer.

Your monorepo has GraphQL schema files (*.graphql) in a dozen packages. They all need the same conventions about naming and deprecation, which are irrelevant to other files. What is the most maintainable way to apply them automatically?

A. A long section in the root CLAUDE.md that applies to everything, relying on Claude to notice GraphQL files.

B. A skill that engineers invoke before editing a schema.

C. A CLAUDE.md in each of the twelve package directories, each containing a copy of the conventions.

D. A .claude/rules/ file with paths: ["**/*.graphql"] that holds the conventions.

Question 26

Select ONE answer.

You must add an export feature, and there are two viable designs: generate files in a background job queue with email delivery, or stream files directly from the web server. They need different infrastructure and have different failure modes. How should you begin in Claude Code?

A. Use plan mode to explore the codebase and evaluate both approaches' implications before choosing and implementing one.

B. Start with direct execution of the simpler streaming approach and switch to a queue only if problems appear in production.

C. Ask for the final code immediately with a detailed prompt describing the streaming design.

D. Implement both designs in parallel with direct execution and keep whichever works first.

Question 27

Select ONE answer.

Claude Code generates DTO mapper boilerplate with inconsistent naming. Your prose rules say 'follow our naming conventions', but results vary. There is also a special case where nullable fields map to Optional types. Which approach is most likely to fix both?

A. Generate the mappers as usual and have a reviewer rename fields after the fact.

B. Provide two or three concrete input/output examples, including one that shows the nullable-to-Optional mapping.

C. Ask Claude to explain in its own words what the naming convention means before generating.

D. Add twenty-five pages of naming guidelines to CLAUDE.md and ask Claude to read them before generating.

Question 28

Select ONE answer.

A developer wants a /standup command that formats their own notes for their personal workflow, not shared with teammates. Where should they create it?

A. In .mcp.json as a server entry named standup.

B. In the project's CLAUDE.md so it applies to their sessions only.

C. In .claude/commands/ in the project repository.

D. In ~/.claude/commands/ in their home directory.

Question 29

Select ONE answer.

A teammate created .claude/rules/python-style.md but forgot to include a paths field. Now every Claude Code session loads these Python rules even when editing Markdown and YAML, wasting tokens and sometimes confusing the output. How should it be fixed?

A. Delete the rule and place its content in the user-level ~/.claude/CLAUDE.md of each developer.

B. Rename the file to python.md so Claude infers from the name that it applies only to Python.

C. Add YAML frontmatter with paths: ["**/*.py"] so the rule loads only when working on matching files.

D. Wrap the rules in a sentence saying 'ignore these unless the file is Python'.

Question 30

Select ONE answer.

Claude generates Express route handlers whose validation and error handling differ from file to file. You have three existing handlers that follow the team's pattern. What is the most effective way to improve consistency?

A. Ask Claude to review each generated handler against the team's written conventions and fix deviations.

B. Generate each handler five times and pick the one that most resembles the team's pattern.

C. Describe the desired pattern in a detailed paragraph, including naming and error rules, and ask Claude to follow it exactly.

D. Include two or three of the existing handlers as examples of the desired structure, validation, and error handling.

Question 31

Select ONE answer.

A developer-productivity agent explores a legacy system in phases: first it maps modules, then it dives into the billing module with new subagents. The new subagents repeat the module mapping because they know nothing about phase one. What is the best fix?

A. Tell each phase-two subagent to read the phase-one agent's conversation to find out what it learned.

B. Summarize the key findings from phase one and inject the summary into the initial prompt of each phase-two subagent.

C. Run phase two in the same session as phase one with no subagents, so it keeps all raw output in context.

D. Skip phase one so the subagents' first task is to map everything they need themselves.

Question 32

Select ONE answer.

An agent explores a large legacy system over several days. At the start of each day it re-explores modules it already analyzed, and its answers sometimes contradict earlier days. What practice best counters this?

A. Disable file writes so that the agent concentrates on keeping the facts in its working memory.

B. Open a new session each day and let it start the analysis from scratch to avoid bias.

C. Have the agent maintain a scratchpad file of key findings (classes, flows, decisions) and consult it at the start of each day and for subsequent questions.

D. Ask the agent to try harder to remember what it learned yesterday and to keep its answers consistent with the conclusions it reached then, whatever they were.

Scenario 5: Claude Code for Continuous Integration (Questions 33-47)

You are integrating Claude Code into your Continuous Integration/Continuous Deployment (CI/CD) pipeline. The system runs automated code reviews, generates test cases, and provides feedback on pull requests. You need to design prompts that provide actionable feedback and minimize false positives.

Question 33

Select ONE answer.

A CI pipeline reviews a 16-file pull request using per-file local passes followed by a cross-file integration pass. The engineer wants to cut review time. Which scheduling is correct?

A. Run the per-file passes and the integration pass all in parallel and merge their outputs at the end.

B. Run the integration pass first so that the per-file passes already know the cross-file concerns.

C. Run everything strictly one after another in file order so each pass sees the previous file's comments.

D. Run the per-file passes in parallel, then run the integration pass afterward using their outputs.

Question 34

Select ONE answer.

Your CI review pipeline always runs all passes (security, performance, test coverage, documentation) on every PR. About 30% of PRs only touch documentation or test fixtures, and the full pipeline wastes time and produces noise on them. What is the better design?

A. Add an initial classification step that identifies the type of change and routes only the relevant passes, defaulting to the full review when the type is unclear.

B. Skip review entirely for any PR whose title contains the word 'docs' or 'fixtures', since those titles almost always describe harmless changes to non-code files.

C. Run the full pipeline anyway but suppress all findings below a certain severity on small PRs.

D. Ask developers to label every PR with its type and run only the passes for that label.

Question 35

Select exactly TWO answers (choose 2).

A single-pass review of a 20-file PR gives deep feedback on some files, shallow feedback on others, and contradictory verdicts on the same pattern in different files. Select the TWO changes that best address this.

A. Switch to a model with a larger context window so that all 20 files can be given the same attention in one pass.

B. Ask developers to resubmit the PR in batches of three or four files before review.

C. Accept only findings that appear in at least two of three full-PR review runs.

D. Split the review into focused per-file passes for local issues plus a separate integration pass for cross-file data flow.

E. Apply the same explicit review criteria and output format to every per-file pass so depth and standards are uniform.

Question 36

Select ONE answer.

In a parallel per-file review, one reviewer subagent hits a transient API timeout while calling a code-search MCP tool. What is the best handling?

A. Abort the entire PR review and fail the CI job so that nothing is posted without full coverage.

B. Retry locally with bounded backoff, and propagate to the coordinator only if it still fails, including what was attempted and any partial results.

C. Propagate every timeout to the coordinator immediately, which then decides whether to retry the call or to give up on that file's review altogether.

D. Ignore the timeout and continue the review of that file without search results, without noting the gap.

Question 37

Select ONE answer.

A developer configures a Jira MCP server in their ~/.claude.json, and Claude Code works well locally. The CI job running Claude Code on the same repo can't use the Jira tools. Which change fixes this properly?

A. Skip MCP in CI and ask Claude to guess issue details from branch names.

B. Define the server in the project's .mcp.json using environment variable expansion for the token, and supply the token to the CI job as a secret.

C. Hard-code the Jira token in .mcp.json so that CI and local use the same literal values.

D. Copy the developer's ~/.claude.json into the repository so that CI can read it from there, and remove the personal entries that are not needed.

Question 38

Select ONE answer.

Your CI step calls Claude Code, which fails once a month because of a temporary API error. The check posts advisory comments, and nothing blocks merges. How should the pipeline handle this failure?

A. Remove the error handling so that failures are ignored silently and never appear in logs.

B. Fail the pipeline immediately on any error from the review step, so that engineers always know the review ran and nobody can merge code that skipped it.

C. Retry a bounded number of times with backoff; if it still fails, mark the advisory check skipped with a clear message rather than blocking the merge.

D. Retry indefinitely until the step succeeds so the review is always posted.

Question 39

Select ONE answer.

A CI run reports three failing checks: a lint error in module A, a type error in module B, and a broken snapshot in module C. They are caused by unrelated changes and do not affect each other. You want each fix verified and committed independently. How should you give feedback to Claude Code to fix them?

A. Ask Claude to ignore two of them and fix only the first, since the others may disappear.

B. Address them one at a time in sequence, since the problems are independent and each fix can be verified separately.

C. Disable the lint, type, and snapshot checks until all modules are stable.

D. Combine all three into one message so Claude sees the whole CI report at once and can plan the fixes together.

Question 40

Select ONE answer.

An auditor asks the team to reproduce exactly how a CI-generated review comment on PR #812 was produced three months ago. Currently nothing but the posted comment is stored. What should the pipeline log?

A. The diff or its commit hash, the prompt and CLAUDE.md versions used, the model and settings, and the raw structured output.

B. Only the final comment text, since that is the artifact that mattered to developers.

C. A screenshot of the pull request page at the time the comment was posted.

D. The names of the developers who reacted to the comment.

Question 41

Select ONE answer.

Your CI test generator mishandles null inputs in a date-parsing helper. You have explained the intended behavior in prose twice. What is the best next step?

A. Provide specific test cases with example inputs and expected outputs for the null and edge cases, and ask Claude to make them pass.

B. Ask Claude to restate the requirement and proceed only when it repeats it correctly.

C. Remove null handling from scope and document that nulls are unsupported.

D. Explain the same behavior a third time with greater emphasis and add exclamation marks to the key requirements, so the expectations are unmistakable.

Question 42

Select ONE answer.

The team wants to decide which CI checks should block a merge. They have deterministic checks (unit tests, type checks, linters) and a Claude review that produces judgment-based findings with some false positives. What is the most sensible policy?

A. Make all checks advisory so developers are never delayed by automation.

B. Make Claude's review the only blocking check, since it understands the code better than static tools do and can catch issues no linter would find.

C. Block on any Claude finding of any severity to be safe.

D. Block merges on the deterministic checks, keep Claude's findings advisory with human approval, and tighten only after measuring precision.

Question 43

Select ONE answer.

A review prompt says 'Look for security issues.' Output includes many speculative remarks and misses clear problems. Which revision best improves precision?

A. List the specific categories to report, such as SQL built from user input, hard-coded secrets, and missing authorization checks, and state which minor items to skip.

B. Add 'only report issues you are highly confident about, and prefer silence over speculation' to the existing prompt so the output is cleaner and more focused for every PR.

C. Remove the security instruction and run a general code review instead.

D. Add 'be thorough but not noisy' to the prompt so that the model balances the two.

Question 44

Select ONE answer.

A team is about to run a 40,000-document batch with an untested prompt. A smaller earlier test showed that about 30% of outputs failed schema validation. Which step best reduces total cost and rework?

A. Move the full run to the synchronous API so failures are discovered one at a time.

B. Split the 40,000 documents into 400 batches of 100 and submit them all at once.

C. Submit the full batch now and fix any failures through resubmission afterward, because batch pricing is cheap and the failures can be handled in bulk.

D. Refine the prompt on a representative sample of a few hundred documents first, then submit the full batch once first-pass success is high.

Question 45

Select ONE answer.

Claude's review labels findings as critical, major, or minor, but the same SQL string-concatenation pattern is 'critical' in one PR and 'minor' in another. How should severity be made consistent?

A. Ask Claude to label severity using its best judgment and be consistent with previous reviews.

B. Remove severity from the output and let developers decide how important each finding is.

C. Define each severity level with explicit criteria and a concrete code example illustrating it.

D. Report the severity as a number from 1 to 100 so small differences average out.

Question 46

Select ONE answer.

Developers dismiss about 40% of Claude's review findings as unhelpful, but nobody can tell which kinds of code trigger the dismissed findings. What schema change would enable systematic analysis?

A. Add a free-form 'comments' field where the model explains its general impressions of the PR and its overall quality in its own words each time.

B. Add a timestamp field so you can see at which hour findings are dismissed most.

C. Add a boolean field helpful that the model sets itself on each finding.

D. Add a detected_pattern field to each finding naming the code construct that triggered it, so dismissals can be analyzed by pattern.

Question 47

Select ONE answer.

The CI job supports three structured outputs depending on PR type: report_findings, report_test_plan, and report_doc_changes. The type isn't known up front, and sometimes Claude replies in prose instead of calling any tool. Which tool_choice setting is best?

A. No tool_choice setting but a stricter system prompt reminding Claude to always call a tool.

B. 'any', which guarantees that some tool is called while letting Claude choose the right one.

C. A forced selection of report_findings for every PR.

D. 'auto', which lets Claude decide whether to call a tool or reply in prose.

Scenario 6: Structured Data Extraction (Questions 48-60)

You are building a structured data extraction system using Claude. The system extracts information from unstructured documents, validates the output using JSON schemas, and maintains high accuracy. It must handle edge cases gracefully and integrate with downstream systems.

Question 48

Select ONE answer.

The extraction agent receives amounts from several parsers as '$1,234.50', '1.234,50 EUR', and '1234.5'. The model sometimes misreads the European format as 1.234. Where is the most reliable place to fix this?

A. A request to the parser vendors to standardize on one format so the problem goes away.

B. A PostToolUse hook that parses each tool result into a decimal amount and ISO currency code before the model sees it.

C. A system prompt paragraph explaining all three formats and asking the model to convert them carefully.

D. A few-shot example in the extraction prompt that shows how to read the European format.

Question 49

Select ONE answer.

Two parallel extraction workers sometimes both create a ledger record for the same invoice number, producing duplicates downstream. Which control is most reliable?

A. A system prompt instruction for each worker to avoid processing invoices that look as if they were already handled by another worker earlier in the run.

B. A nightly job that deletes duplicate records after they have been created.

C. A pre-call hook or tool-level check on create_record that enforces uniqueness atomically on the invoice's natural key (for example a unique constraint) and redirects duplicates.

D. A requirement that the model explain in its response why it believes the invoice is new.

Question 50

Select ONE answer.

A PostToolUse hook normalizes dates and amounts before the model sees OCR results. An auditor now asks how a reported total relates to the original OCR text, and the raw text is gone. How should the design change?

A. Have the model write the original text into its response alongside every normalized field.

B. Keep the hook, but store the original raw tool output with a reference ID alongside the normalized values so each field can be traced to its source.

C. Keep only the normalized values and tell the auditor that normalization is deterministic.

D. Remove the hook and let the model see the raw OCR text again, so there is only a single version of the data and nothing to reconcile for the auditor later.

Question 51

Select ONE answer.

Which description for an extract_invoice tool is most likely to produce reliable tool selection among similar tools?

A. 'Extracts structured data from documents such as invoices, receipts, statements, and other financial paperwork submitted by users, and returns the results as clean JSON that is ready for downstream use.'

B. 'Used for invoices. See the documentation page for more details about supported file types and options.'

C. 'Extracts header fields (vendor, invoice number, date, currency) and line items from a vendor invoice or credit note PDF. Do not use for point-of-sale receipts (use extract_receipt) or contracts.'

D. 'A powerful and flexible tool for extracting all kinds of information from business documents quickly and accurately.'

Question 52

Select ONE answer.

You submit a batch of 1,000 extraction requests. When results arrive, a developer assumes they are in the same order as submitted and joins them to the source rows by position. Some records are joined to the wrong documents. What should have been used?

A. The timestamp at which each result was returned, sorted ascending.

B. The length of each extracted text, used to find the matching document.

C. Relying on the order in which requests were submitted, since the service returns results in that order.

D. A unique custom_id on each request, used to correlate each result to its source document.

Question 53

Select ONE answer.

Source documents write dates as '3/4/26', 'March 4, 2026', and '4 Mar 26', and sometimes omit them. The schema has an ISO date field. What combination best yields clean, honest output?

A. A schema with no date field, since dates are unreliable in documents.

B. A nullable date field plus prompt normalization rules (convert to ISO 8601, return null if no date appears) alongside the strict schema.

C. A required date field and a prompt telling the model to infer a plausible date from context whenever none is present in the document, so that the field is never empty.

D. A free-text date field so the model can copy exactly what the document says.

Question 54

Select exactly TWO answers (choose 2).

A validation-retry loop attaches the specific validation error to follow-up requests. Select the TWO failures where this retry is most likely to succeed.

A. A vendor tax ID the schema requires, but which does not appear anywhere in the document.

B. A scanned page that is too blurry for any reader to make out the amount.

C. A line item missing its quantity although the quantity is plainly printed in the document.

D. A purchase order number that exists only in a separate document not provided to the model.

E. A total returned as the string '1,234.50' that fails a numeric type check.

Question 55

Select ONE answer.

Your extraction service has two workloads: (1) urgent invoices that staff are waiting on, with results needed within minutes, and (2) an archive of 200,000 historical documents to process over the next two weeks. Which API strategy is best?

A. Synchronous API for the urgent invoices, and the Message Batches API for the archive.

B. Synchronous API for both, because consistent behavior is easier to reason about.

C. Message Batches API for both, with a polling loop for the urgent invoices to check progress.

D. Message Batches API for the archive, and for urgent invoices submit one-item batches.

Question 56

Select ONE answer.

A developer is new to the Message Batches API and describes a plan: submit the batch, then wait. Which description of the correct workflow is most accurate?

A. Submit the batch with a custom_id per request, poll for completion (results may take up to 24 hours), then retrieve results and correlate by custom_id.

B. Submit the batch and expect results within 30 seconds, with a guaranteed latency SLA.

C. Submit the batch and receive all results immediately in the same response, since batches are synchronous, which then need to be correlated with the requests by their order.

D. Submit each request one at a time and block until each completes before sending the next.

Question 57

Select exactly TWO answers (choose 2).

A reviewer team can handle 200 documents per day out of 5,000 extracted. Select the TWO practices that best prioritize their attention.

A. Review the 200 documents with the longest text, since longer documents have more errors.

B. Review a fixed 4% chosen alphabetically by vendor name.

C. Have the model output field-level confidence and set review thresholds by calibrating against a labeled validation set.

D. Route extractions whose sources are ambiguous or contradictory to human review first.

E. Trust the model's overall self-rated confidence for each document, uncalibrated, and review the lowest 200.

Question 58

Select ONE answer.

An extraction pipeline stores, for every field, the model-provided supporting quote and page number so reviewers can verify values quickly. A spot check finds some quotes are paraphrases that do not appear in the document. What is the best control?

A. Verify programmatically that each quote is an exact substring of the source text at the stated page, and flag mismatches for review or retry.

B. Accept paraphrases as long as the meaning matches the source.

C. Ask the model to confirm in a follow-up message that each of its quotes is accurate and appears verbatim in the source document at the page it cited.

D. Drop the quote field because paraphrased quotes cause confusion.

Question 59

Select ONE answer.

An analyst extracts 'total assets' from two versions of the same filing: the original (2024-03-15) and an amended version (2024-09-02). The values differ slightly. The pipeline flags a contradiction. What should the structured output include so that the downstream reader can interpret the difference correctly?

A. A boolean marking the larger value as correct.

B. A note saying the values are 'approximately the same'.

C. Only the most recently ingested value, with the earlier one deleted.

D. The document date or collection date for each value, so the later amendment is recognized as a revision rather than a conflict.

Question 60

Select ONE answer.

Per-field confidence scores come out of the extraction model. The team wants to auto-accept fields above a threshold and send the rest to reviewers, who can handle limited volume. How should they choose the threshold?

A. Pick the threshold that sends exactly as many documents to review as reviewers can handle in a day, and accept whatever accuracy results in the auto-accepted set at that level.

B. Ask the model what threshold it believes is appropriate and use its answer.

C. Choose 0.9 because it is a common default and works well in many systems.

D. Use a labeled validation set to measure accuracy at different confidence levels, per document type and field, and pick the threshold meeting the target within reviewer capacity.

End of examination. Stop here, then check your answers using the key that follows.

5. Quick answer key and score tracker

Mark each question you answered correctly (all required selections, no wrong ones). Then complete the domain tracker below.

No.	Ans.	Domain	Scen.	Mine
1	**A**	D1	S3	
2	**B**	D1	S3	
3	**A**	D1	S3	
4	**A**	D1	S3	
5	**B**	D1	S3	
6	**D**	D1	S3	
7	**C**	D2	S3	
8	**B,E**	D2	S3	
9	**C**	D2	S3	
10	**B**	D2	S3	
11	**B**	D2	S3	
12	**C**	D4	S3	
13	**C,D**	D5	S3	
14	**A,D,E**	D5	S3	
15	**A**	D5	S3	
16	**C**	D1	S4	
17	**A**	D1	S4	
18	**B**	D1	S4	
19	**C**	D1	S4	
20	**C**	D2	S4	
21	**C**	D2	S4	
22	**D**	D2	S4	
23	**D**	D3	S4	
24	**A**	D3	S4	
25	**D**	D3	S4	
26	**A**	D3	S4	
27	**B**	D3	S4	
28	**D**	D3	S4	
29	**C**	D3	S4	
30	**D**	D4	S4	

No.	Ans.	Domain	Scen.	Mine
31	**B**	D5	S4	
32	**C**	D5	S4	
33	**D**	D1	S5	
34	**A**	D1	S5	
35	**D,E**	D1	S5	
36	**B**	D2	S5	
37	**B**	D2	S5	
38	**C**	D3	S5	
39	**B**	D3	S5	
40	**A**	D3	S5	
41	**A**	D3	S5	
42	**D**	D3	S5	
43	**A**	D4	S5	
44	**D**	D4	S5	
45	**C**	D4	S5	
46	**D**	D4	S5	
47	**B**	D4	S5	
48	**B**	D1	S6	
49	**C**	D1	S6	
50	**B**	D1	S6	
51	**C**	D2	S6	
52	**D**	D4	S6	
53	**B**	D4	S6	
54	**C,E**	D4	S6	
55	**A**	D4	S6	
56	**A**	D4	S6	
57	**C,D**	D5	S6	
58	**A**	D5	S6	
59	**D**	D5	S6	
60	**D**	D5	S6	

Domain-level score tracker

Domain	Questions	Number list	My correct	% correct
1. Agentic Architecture & Orchestration	16	1, 2, 3, 4, 5, 6, 16, 17, 18, 19, 33, 34, 35, 48, 49, 50		
2. Tool Design & MCP Integration	11	7, 8, 9, 10, 11, 20, 21, 22, 36, 37, 51		
3. Claude Code Configuration & Workflows	12	23, 24, 25, 26, 27, 28, 29, 38, 39, 40, 41, 42		
4. Prompt Engineering & Structured Output	12	12, 30, 43, 44, 45, 46, 47, 52, 53, 54, 55, 56		
5. Context Management & Reliability	9	13, 14, 15, 31, 32, 57, 58, 59, 60		
Total	**60**			

Interpretation: percent-correct here is a practice signal only. The real exam's pass/fail decision is based on a single scaled score (cut score 720 of 1,000); domain percentages on the real score report are informational.

6. Detailed answer key and explanations

Each entry gives the correct answer, why it is correct, why every other option is wrong, the concept tested, the detail in the scenario that decides the answer, and a production lesson.

Q1 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.1 | Challenging | Single answer

Correct answer: A - Handle stop_reason explicitly: treat 'end_turn' as complete, and for 'max_tokens' do not ship the output as final but continue generation or retry with a higher output limit.

Why the correct answer is correct: The loop should branch on stop_reason: 'tool_use' means run the tools and continue, 'end_turn' means the model considers the answer complete, and 'max_tokens' means the output was cut off and must be handled rather than delivered as final. Inferring completion from message text or treating all other values the same masks truncation.

Option-by-option:

A. **[CORRECT]** Handle stop_reason explicitly: treat 'end_turn' as complete, and for 'max_tokens' do not ship the output as final but continue generation or retry with a higher output limit.
Correct. A truncated response is not a completed one, and the loop must distinguish the cases.

B. **[INCORRECT]** Keep treating every non-tool_use response as final, but add a check that the text ends with a period or other closing punctuation before the report is returned to the user, and retry once if it does not.
A trailing period is a weak text heuristic that truncated output can satisfy, and it is still inferring completion from content.

C. **[INCORRECT]** Raise the iteration cap so the loop gets more chances to finish the report before it stops.
The loop is not hitting an iteration cap. It is treating a truncated response as complete after one pass.

D. **[INCORRECT]** Instruct the model in the system prompt to keep each report short enough that it never reaches the output limit.
Brevity instructions are not guaranteed and large reports will still be truncated sometimes.

Key concept: Complete stop_reason handling in the agentic loop (Task 1.1).
Exam reasoning: The logs name the actual stop_reason of the failing responses, which tells you the loop's else-branch is too broad.
Production lesson: Handle every documented stop_reason deliberately. An 'else' branch that means 'done' is a bug waiting to happen.

Q2 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.2 | Foundational | Single answer

Correct answer: B - In the coordinator, which sees every subagent's results and is responsible for aggregation and reconciling overlap before synthesis.

Why the correct answer is correct: In a hub-and-spoke design, the coordinator handles task decomposition, delegation, result aggregation, and information routing. Because subagents operate with isolated context, only the coordinator can compare their outputs.

Option-by-option:

A. **[INCORRECT]** In the report-generation subagent, which should detect duplicates while formatting the final document.
By then the evidence has already been counted as independent and synthesis has been done.

B. **[CORRECT]** In the coordinator, which sees every subagent's results and is responsible for aggregation and reconciling overlap before synthesis.
Correct. Aggregation across subagents is a coordinator responsibility.

C. **[INCORRECT]** In each subagent, by instructing each one to check whether either of the other two might already have found the same source and to skip it if so.
Subagents have isolated context and cannot see each other's results.

D. **[INCORRECT]** In the user's review step, by asking readers to flag repeated citations after the report is delivered.
This pushes a system responsibility onto the reader.

Key concept: Coordinator responsibilities (Task 1.2).
Exam reasoning: Only one component sees all three result sets before synthesis.
Production lesson: Put cross-agent concerns in the agent that has cross-agent visibility.

Q3 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.3 | Intermediate | Single answer, secondary: D2

Correct answer: A - Define the subagent with a specific description of its role, a focused system prompt, and a restricted tool list limited to search and fetch tools.

Why the correct answer is correct: An AgentDefinition includes a description (which guides delegation decisions), a system prompt, and tool restrictions. Scoping tools to the role prevents cross-specialization misuse.

Option-by-option:

A. **[CORRECT]** Define the subagent with a specific description of its role, a focused system prompt, and a restricted tool list limited to search and fetch tools.
Correct. AgentDefinition fields describe when to use it and constrain what it can do.

B. **[INCORRECT]** Rename the subagent to 'WebResearcher' and leave the description and tool list unchanged.
A better name alone does not restrict tools or explain when to use the agent.

C. **[INCORRECT]** Keep the broad tool list but add a line to the coordinator's prompt telling it which tasks should never be sent to the web researcher.
This leaves the capability to overwrite files in place and depends on the coordinator following instructions.

D. **[INCORRECT]** Remove the subagent and have the coordinator perform web research itself so there is no delegation to get wrong.
This gives up specialization and puts research tools on the coordinator.

Key concept: AgentDefinition configuration (Task 1.3, 2.3).
Exam reasoning: Both symptoms, misdelegation and an unintended write, are addressed by definition-level description and tool restrictions.
Production lesson: Constrain subagents by configuration, not by hoping they behave.

Q4 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.3 | Intermediate | Single answer, secondary: D5

Correct answer: A - Structured records that separate each claim's content from its metadata: source document, page number, URL, and publication date.

Why the correct answer is correct: When passing context between agents, use structured data formats that separate content from metadata such as source URLs, document names, and page numbers, so attribution is preserved.

Option-by-option:

A. **[CORRECT]** Structured records that separate each claim's content from its metadata: source document, page number, URL, and publication date.
Correct. Structured separation keeps metadata attached through the handoff.

B. **[INCORRECT]** Short summaries without any metadata so the synthesis agent can focus on content.
Removing metadata guarantees that attribution is lost.

C. **[INCORRECT]** A single combined paragraph with all claims followed by a bibliography of every document consulted.
A bibliography doesn't map individual claims to sources.

D. **[INCORRECT]** Longer free-form paragraphs where the analyst mentions the source document name and page at the start of each paragraph, so the writer can see them.
Names in prose are easily lost or blended when the synthesis agent rewrites.

Key concept: Structured context passing (Task 1.3, 5.6).
Exam reasoning: The symptom is lost attribution at a handoff, which structure addresses.
Production lesson: Treat metadata as first-class fields in every handoff.

Q5 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.1 | Intermediate | Single answer

Correct answer: B - Let Claude decide which tool to call next from the context and results so far, within the agentic loop, with guardrails for cost and safety.

Why the correct answer is correct: Agentic loops are model-driven: Claude reasons about which tool to call next based on context, rather than following pre-configured decision trees or tool sequences.

Option-by-option:

A. **[INCORRECT]** Keep the fixed sequence but increase it to five searches, so that complex queries are covered as well, and skip the fifth when the first four agree.
A larger fixed sequence wastes work on simple queries and still cannot follow up on discoveries.

B. **[CORRECT]** Let Claude decide which tool to call next from the context and results so far, within the agentic loop, with guardrails for cost and safety.
Correct. Model-driven decisions adapt to what each result reveals.

C. **[INCORRECT]** Run a classifier model to predict how many searches each query will need and schedule exactly that many.
A prediction before research begins can't adjust when results change the plan.

D. **[INCORRECT]** Add a decision tree in code that checks keywords in the query and selects among ten pre-defined sequences.
Keyword trees can't anticipate intermediate findings and are brittle.

Key concept: Model-driven versus pre-configured control flow (Task 1.1).
Exam reasoning: The need for follow-ups based on earlier results is what a fixed sequence cannot provide.
Production lesson: Let the model route within the loop. Put hard rules in hooks.

Q6 | Domain 1: Agentic Architecture & Orchestration | Scenario 3: Multi-Agent Research System

Task statement 1.2 | Challenging | Single answer

Correct answer: D - Improve the coordinator's decomposition by prompting it to enumerate the topic's dimensions (technologies, regions, policy, finance) and add a coverage review that re-delegates gaps.

Why the correct answer is correct: Overly narrow task decomposition by the coordinator leads to incomplete coverage. Coordinators should partition scope across subagents and use iterative refinement: evaluate synthesis for gaps and re-delegate with targeted queries.

Option-by-option:

A. **[INCORRECT]** Run the same single subtask five times with different search engines to gather more material.
More searches on the same narrow subtask do not widen the scope.

B. **[INCORRECT]** Instruct the web search subagent to broaden its queries beyond the assigned subtask whenever it feels a topic is incomplete, and to report on anything else that it finds along the way.
Subagents should work within their assignment. Scope is set by the coordinator.

C. **[INCORRECT]** Add more synthesis instructions telling the synthesis subagent to highlight everything the report is missing.
Synthesis can only describe the findings it receives. It cannot supply research that was never done.

D. **[CORRECT]** Improve the coordinator's decomposition by prompting it to enumerate the topic's dimensions (technologies, regions, policy, finance) and add a coverage review that re-delegates gaps.
Correct. The root cause is narrow decomposition, and the fix belongs in the coordinator.

Key concept: Coordinator decomposition quality and refinement (Task 1.2).
Exam reasoning: Subagents succeeded within their narrow assignment, so the defect lies upstream.
Production lesson: Check the plan before you blame the workers.

Q7 | Domain 2: Tool Design & MCP Integration | Scenario 3: Multi-Agent Research System

Task statement 2.4 | Intermediate | Single answer

Correct answer: C - Expose the collection catalog as an MCP resource and provide the query and fetch operations as tools.

Why the correct answer is correct: MCP resources expose content catalogs (documentation hierarchies, schemas, summaries) to reduce exploratory tool calls, while tools are for actions.

Option-by-option:

A. **[INCORRECT]** Put the collection catalog in the system prompt and remove the listing capability from the server.
The catalog may change and would then have to be maintained in two places.

B. **[INCORRECT]** Expose all three as resources so the agent can read them whenever it needs.
Queries take parameters and perform work, which suits tools rather than resources.

C. **[CORRECT]** Expose the collection catalog as an MCP resource and provide the query and fetch operations as tools.
Correct. Resources give visibility into available content and tools perform actions.

D. **[INCORRECT]** Expose all three as tools and have the agent call the listing tool before every query.
The catalog is static reference content, so repeated tool calls add latency and overhead.

Key concept: Resources for catalogs, tools for actions (Task 2.4).
Exam reasoning: The three capabilities split naturally into one catalog and two operations.
Production lesson: Resources describe what exists; tools do things.

Q8 | Domain 2: Tool Design & MCP Integration | Scenario 3: Multi-Agent Research System

Task statement 2.3 | Intermediate | Select 2

Correct answer: B, E - Give the synthesis subagent a narrow verify_fact tool for simple lookups and route complex verification through the coordinator. / Give the web search subagent only search and load-document tools and keep file-writing tools with the report generator.

Why the correct answer is correct: Restrict each subagent's tool set to its role, provide scoped cross-role tools for high-frequency needs, and route complex cases through the coordinator.

Option-by-option:

A. **[INCORRECT]** Give the synthesis subagent unrestricted web search so it never needs to wait for the coordinator.
This over-provisions the synthesis agent and blurs responsibilities.

B. **[CORRECT]** Give the synthesis subagent a narrow verify_fact tool for simple lookups and route complex verification through the coordinator.
Correct. A scoped cross-role tool covers a frequent need without over-provisioning.

C. **[INCORRECT]** Give every subagent the full tool set so that any of them can complete any task without delegation.
Too many tools degrade selection and encourage out-of-role behavior.

D. **[INCORRECT]** Remove all tools from subagents and have the coordinator execute every tool call on their behalf.
This makes the coordinator a bottleneck and removes specialization.

E. **[CORRECT]** Give the web search subagent only search and load-document tools and keep file-writing tools with the report generator.
Correct. Each agent gets the tools its role requires.

Key concept: Scoped tool access (Task 2.3).
Exam reasoning: Both correct choices give narrow capability; both wrong choices give broad capability.
Production lesson: Least privilege improves safety and tool-selection reliability.

Q9 | Domain 2: Tool Design & MCP Integration | Scenario 3: Multi-Agent Research System

Task statement 2.1 | Intermediate | Single answer

Correct answer: C - Split it into purpose-specific tools such as extract_data_points, summarize_content, and verify_claim_against_source, each with a defined input and output contract.

Why the correct answer is correct: Splitting generic tools into purpose-specific tools with defined input and output contracts improves reliability, for example separating extraction, summarization, and verification.

Option-by-option:

A. **[INCORRECT]** Keep one tool and add a free-text 'output_mode' parameter in which the model writes whatever format it wants, so that each caller can ask for the shape it needs from the document at that moment.
A free-text mode keeps the contract ambiguous.

B. **[INCORRECT]** Keep one tool and ask downstream agents to detect the output shape and adapt their parsing.
This moves the problem downstream instead of fixing the contract.

C. **[CORRECT]** Split it into purpose-specific tools such as extract_data_points, summarize_content, and verify_claim_against_source, each with a defined input and output contract.
Correct. Narrow tools produce predictable outputs and clearer selection.

D. **[INCORRECT]** Remove the tool and let the subagent answer from its own knowledge to avoid format problems.
That removes grounding in the actual documents.

Key concept: Splitting generic tools (Task 2.1).
Exam reasoning: One tool producing three shapes is the sign that it does too many jobs.
Production lesson: One tool, one job, one contract.

Q10 | Domain 2: Tool Design & MCP Integration | Scenario 3: Multi-Agent Research System

Task statement 2.2 | Foundational | Single answer

Correct answer: B - Shorten or rephrase the query to meet the limit and call the tool again, since the error is caused by the input rather than a temporary outage.

Why the correct answer is correct: Error categories guide recovery: transient errors may be retried, validation errors need corrected input, business errors reflect policy, and permission errors need authorization changes.

Option-by-option:

A. **[INCORRECT]** Report to the user that the search service is down and stop researching this topic.
The service is working, and the agent can recover by fixing the query.

B. **[CORRECT]** Shorten or rephrase the query to meet the limit and call the tool again, since the error is caused by the input rather than a temporary outage.

Correct. A validation error is fixed by correcting the input.

C. **[INCORRECT]** Retry the identical call after a short delay with exponential backoff, as most tool errors clear up on their own once the service has recovered.
The same oversized query will fail again, because the cause is not temporary.

D. **[INCORRECT]** Escalate to a human reviewer as a permission problem and wait for access to be granted.
This is not a permission failure, and no human needs to be involved.

Key concept: Error categories and recovery (Task 2.2).
Exam reasoning: The category 'validation' tells you the input is the problem.
Production lesson: Use the error category to choose the recovery, not a reflex.

Q11 | Domain 2: Tool Design & MCP Integration | Scenario 3: Multi-Agent Research System

Task statement 2.3 | Foundational | Single answer

Correct answer: B - Replace fetch_url with a load_document tool that validates the URL against allowed document sources and types before fetching.

Why the correct answer is correct: Replace generic tools with constrained alternatives that validate inputs, such as load_document in place of fetch_url, to apply least privilege.

Option-by-option:

A. **[INCORRECT]** Keep fetch_url and tell the subagent in its prompt to fetch only research documents and avoid anything internal.
A prompt cannot guarantee that the model never requests an unwanted URL.

B. **[CORRECT]** Replace fetch_url with a load_document tool that validates the URL against allowed document sources and types before fetching.
Correct. A constrained tool provides the needed capability with guardrails.

C. **[INCORRECT]** Give the subagent access to the coordinator's web search tools instead of fetch_url so it can find documents itself.
This broadens its capabilities rather than constraining them.

D. **[INCORRECT]** Log every fetched URL and review the logs weekly for anything suspicious.
Logging helps detect problems but doesn't prevent them.

Key concept: Constrained tools (Task 2.3).
Exam reasoning: The risk is the tool's generality, so the fix is to narrow it.
Production lesson: Constrain at the tool, not in the prompt.

Q12 | Domain 4: Prompt Engineering & Structured Output | Scenario 3: Multi-Agent Research System

Task statement 4.6 | Intermediate | Single answer, secondary: D5

Correct answer: C - Have the verifier output a confidence value with each finding, calibrate thresholds on a labeled validation set, and route low-confidence ones to editors.

Why the correct answer is correct: Verification passes where the model self-reports confidence with each finding enable calibrated review routing, and thresholds should be calibrated using labeled validation data.

Option-by-option:

A. **[INCORRECT]** Send editors a random sample of 30 claims from each report, and assume that the remaining claims follow the same quality as the ones that were sampled, adjusting the sample size over time.
Random sampling ignores which claims are risky.

B. **[INCORRECT]** Ask the verifier to rate its overall confidence for the whole report and review the report only if the rating is below 5.
A single score hides which claims need attention.

C. **[CORRECT]** Have the verifier output a confidence value with each finding, calibrate thresholds on a labeled validation set, and route low-confidence ones to editors.
Correct. Calibrated confidence enables routing review attention.

D. **[INCORRECT]** Treat every claim the verifier marks as supported as final and send all unsupported claims to editors, regardless of count.
This ignores editor capacity and the verifier's own errors.

Key concept: Confidence-based routing (Task 4.6, 5.5).
Exam reasoning: Limited editor capacity creates the need to prioritize, and calibration prevents trusting raw numbers.
Production lesson: Use confidence to prioritize, and calibrate it before relying on it.

Q13 | Domain 5: Context Management & Reliability | Scenario 3: Multi-Agent Research System

Task statement 5.6 | Intermediate | Select 2

Correct answer: C, D - Present both values with their sources and an explanation of the conflict instead of silently choosing one. / Require publication or collection dates in subagent outputs so differences can be interpreted as timing rather than contradiction.

Why the correct answer is correct: Annotate conflicts with source attribution rather than arbitrarily selecting one value, and require dates so temporal differences aren't misinterpreted as contradictions.

Option-by-option:

A. **[INCORRECT]** Choose whichever source is cited most often in the other documents.
Citation frequency doesn't resolve differences in dates or methodology.

B. **[INCORRECT]** Report the average of the two figures as the best estimate.
Averaging manufactures a number that no source reported.

C. **[CORRECT]** Present both values with their sources and an explanation of the conflict instead of silently choosing one.
Correct. Conflicts are annotated with attribution.

D. **[CORRECT]** Require publication or collection dates in subagent outputs so differences can be interpreted as timing rather than contradiction.
Correct. Dates prevent temporal differences from being mistaken for conflicts.

E. **[INCORRECT]** Silently use the most recent figure and omit the older one to keep the report simple.
Dropping one value hides the discrepancy and might be wrong if methodologies differ.

Key concept: Conflicting statistics and temporal data (Task 5.6).
Exam reasoning: The two sources differ in date, so both dates and attribution matter.
Production lesson: Preserve disagreement and its context.

Q14 | Domain 5: Context Management & Reliability | Scenario 3: Multi-Agent Research System

Task statement 5.3 | Intermediate | Select 3

Correct answer: A, D, E - Return structured error context to the coordinator: failure type, attempted query, partial results, and alternatives. / Annotate the final synthesis with coverage information stating which topics are well supported and which have gaps. / Retry locally for the transient failure and propagate only if it cannot be resolved.

Why the correct answer is correct: Subagents should recover locally from transient failures and propagate unresolved errors with structured context and partial results; synthesis should carry coverage annotations. Generic statuses, workflow termination, and silent suppression are anti-patterns.

Option-by-option:

A. **[CORRECT]** Return structured error context to the coordinator: failure type, attempted query, partial results, and alternatives.
Correct. This enables intelligent recovery decisions.

B. **[INCORRECT]** Return a generic status such as 'analysis unavailable' so the coordinator is not confused by details.
Generic statuses hide information needed for recovery.

C. **[INCORRECT]** Terminate the whole research workflow on the first error to avoid incomplete reports.
This discards valid work from other subagents.

D. **[CORRECT]** Annotate the final synthesis with coverage information stating which topics are well supported and which have gaps.
Correct. Readers see what the evidence does and does not cover.

E. **[CORRECT]** Retry locally for the transient failure and propagate only if it cannot be resolved.
Correct. Local recovery avoids unnecessary coordinator involvement.

F. **[INCORRECT]** Return an empty analysis marked successful so the pipeline continues smoothly.
This suppresses the error and misleads downstream agents.

Key concept: Error propagation strategy (Task 5.3).
Exam reasoning: Each correct option supports a different stage of recovery: local, coordinator, and report.
Production lesson: Fail informatively and degrade gracefully.

Q15 | Domain 5: Context Management & Reliability | Scenario 3: Multi-Agent Research System

Task statement 5.3 | Foundational | Single answer

Correct answer: A - Results for the 8 readable documents, plus a structured error for each of the 2 failures stating what was attempted and why it failed.

Why the correct answer is correct: Return partial results with structured error context for what failed, enabling the coordinator to proceed and annotate coverage rather than hiding or over-reacting to failures.

Option-by-option:

A. **[CORRECT]** Results for the 8 readable documents, plus a structured error for each of the 2 failures stating what was attempted and why it failed.
Correct. Partial success is delivered with explicit, actionable failure information.

B. **[INCORRECT]** Results for the 8 readable documents only, with no mention of the others so the output stays clean.
Silently omitting documents hides a coverage gap.

C. **[INCORRECT]** A single failure for the whole job with no partial results, because a partial analysis could mislead the coordinator about what the corpus contains.
This discards 8 valid results unnecessarily.

D. **[INCORRECT]** Empty analyses for the 2 corrupted documents marked as successful, to keep the output format uniform.
Fake successes misrepresent what was analyzed.

Key concept: Partial results with error context (Task 5.3).
Exam reasoning: Two of ten failing is a partial failure, which should be reported as such.
Production lesson: Report what worked and what didn't, side by side.

Q16 | Domain 1: Agentic Architecture & Orchestration | Scenario 4: Developer Productivity with Claude

Task statement 1.3 | Challenging | Single answer

Correct answer: C - Give each explorer a distinct set of modules, include a shared baseline summary in each prompt, and spawn them together with multiple Task calls in one response.

Why the correct answer is correct: Partition research or exploration scope across subagents to minimize duplication, pass needed context explicitly because subagents do not inherit it, and spawn parallel subagents by emitting multiple Task calls in a single response.

Option-by-option:

A. **[INCORRECT]** Keep three identical explorers but have the coordinator discard the two shortest summaries.
Discarding results wastes the cost of producing them and does not widen coverage.

B. **[INCORRECT]** Reduce to one explorer and have it map the whole repository sequentially to avoid any overlap.
That avoids duplication but loses parallelism and risks exhausting a single agent's context.

C. **[CORRECT]** Give each explorer a distinct set of modules, include a shared baseline summary in each prompt, and spawn them together with multiple Task calls in one response.
Correct. Partitioned scope reduces duplication, explicit context replaces assumed sharing, and parallel spawning cuts latency.

D. **[INCORRECT]** Keep the same instruction for all three, but ask each explorer to read the others' output files as they finish and to skip any area already covered by someone else.
Subagents run in isolation and finish at different times, so they cannot coordinate that way.

Key concept: Partitioning, explicit context, and parallel spawning (Task 1.3, 1.2).
Exam reasoning: Overlap without awareness of one another shows missing partitioning, and separate turns show missing parallelism.
Production lesson: Divide the work, hand each worker what it needs, and launch together.

Q17 | Domain 1: Agentic Architecture & Orchestration | Scenario 4: Developer Productivity with Claude

Task statement 1.2 | Challenging | Single answer, secondary: D5

Correct answer: A - Task 3, because the services are independent, the exploration is verbose, and isolating each lets the coordinator compare compact results.

Why the correct answer is correct: Multiple agents add coordination cost and handoff risk. They are justified when work is independent, verbose, or parallelizable and isolation preserves the coordinator's context.

Option-by-option:

A. **[CORRECT]** Task 3, because the services are independent, the exploration is verbose, and isolating each lets the coordinator compare compact results.
Correct. Independence, volume, and a comparison step are the conditions where subagents pay off.

B. **[INCORRECT]** Task 1, because delegating even a small edit to a subagent isolates it from the main context and keeps that context clean.
A one-file rename gains nothing from delegation and adds overhead.

C. **[INCORRECT]** Task 2, because explanation is easier for a specialized explainer agent than for the main agent.
A single function is well within one agent's ability.

D. **[INCORRECT]** Task 4, because tightly coupled edits benefit from splitting the lines among several agents.
Tight coupling makes splitting harmful, since the edits must be consistent.

Key concept: Single versus multi-agent decisions (Task 1.2, 5.4).
Exam reasoning: Only one option has independent parts and a need to compare condensed results.
Production lesson: Use more agents only when the structure of the work calls for it.

Q18 | Domain 1: Agentic Architecture & Orchestration | Scenario 4: Developer Productivity with Claude

Task statement 1.7 | Challenging | Single answer

Correct answer: B - Start a new session seeded with a structured summary of the conclusions that still hold, and re-explore the parts that changed.

Why the correct answer is correct: Starting a new session with a structured summary is more reliable than resuming with stale tool results. Resumption fits when prior context is mostly valid, with changes communicated for targeted re-analysis.

Option-by-option:

A. **[INCORRECT]** Resume the session and tell it only that 'some files changed', then continue using its earlier findings as they are.
With 60% of the files affected, vague notice leaves the agent relying on many stale results.

B. **[CORRECT]** Start a new session seeded with a structured summary of the conclusions that still hold, and re-explore the parts that changed.
Correct. Most earlier tool results are stale, so a summary plus targeted re-exploration is more reliable.

C. **[INCORRECT]** Fork the session so the stale analysis becomes the baseline of a new branch and continue from there.
A fork inherits the same stale history.

D. **[INCORRECT]** Delete the notes and ask the agent to answer from general knowledge of the module's purpose.
That discards valid conclusions and invites guessing.

Key concept: Resume versus fresh start (Task 1.7).
Exam reasoning: The share of affected files (60%) pushes toward starting fresh with a summary.
Production lesson: The more of the history that has changed, the less it is worth resuming.

Q19 | Domain 1: Agentic Architecture & Orchestration | Scenario 4: Developer Productivity with Claude

Task statement 1.2 | Intermediate | Single answer

Correct answer: C - Evaluate the report against the goal, identify the webhook gap, and re-delegate a targeted exploration with a specific query before finalizing.

Why the correct answer is correct: Iterative refinement loops let the coordinator evaluate output for gaps and re-delegate with targeted queries until coverage is sufficient.

Option-by-option:

A. **[INCORRECT]** Have the explorer guess how webhooks typically work in payment systems and add a paragraph.
Guessing produces unverified claims about this specific codebase.

B. **[INCORRECT]** Ask the explorer to expand the whole report to be more comprehensive in general and resubmit it.
A vague request does not target the known gap.

C. **[CORRECT]** Evaluate the report against the goal, identify the webhook gap, and re-delegate a targeted exploration with a specific query before finalizing.
Correct. Iterative refinement lets the coordinator close coverage gaps.

D. **[INCORRECT]** Accept the report and tell the user that webhooks were treated as out of scope for this run, offering to investigate them in a follow-up request.
The user asked about webhooks, so leaving them out fails the task.

Key concept: Coordinator iterative refinement (Task 1.2).
Exam reasoning: The coordinator holds the user's goal and can see the report is missing a central topic.
Production lesson: Evaluate returned work against the goal and follow up precisely.

Q20 | Domain 2: Tool Design & MCP Integration | Scenario 4: Developer Productivity with Claude

Task statement 2.5 | Foundational | Single answer

Correct answer: C - Edit, replacing the unique text 'timeout = 30' with 'timeout = 60'.

Why the correct answer is correct: Edit performs targeted modifications using unique text matching; Write is for full file operations; Grep searches content; Glob matches paths.

Option-by-option:

A. **[INCORRECT]** Glob, with a pattern that matches 'timeout = 30' and replaces it.
Glob matches file paths, not file contents.

B. **[INCORRECT]** Grep, with a replacement option to change the value in place.
Grep searches contents but is not for modifying files.

C. **[CORRECT]** Edit, replacing the unique text 'timeout = 30' with 'timeout = 60'.
Correct. Edit makes targeted changes using unique text matching.

D. **[INCORRECT]** Write, regenerating all 2,000 lines of the file with the new value.
Rewriting the whole file risks accidental changes and is unnecessary.

Key concept: Selecting the right built-in tool (Task 2.5).
Exam reasoning: The text is unique, which is the condition Edit requires.
Production lesson: Prefer the smallest tool that does the job.

Q21 | Domain 2: Tool Design & MCP Integration | Scenario 4: Developer Productivity with Claude

Task statement 2.4 | Challenging | Single answer

Correct answer: C - Tools from all configured servers are available at once, so differentiate the two tool descriptions (or remove the personal tool) to eliminate the overlap.

Why the correct answer is correct: Tools from all configured MCP servers are discovered at connection time and available simultaneously. Overlapping tool descriptions cause misrouting.

Option-by-option:

A. **[INCORRECT]** The project server always takes priority over the user server, so the personal tool will never be chosen.
There is no such precedence that prevents selection of either tool.

B. **[INCORRECT]** Add the personal server to the project's .mcp.json so that both tools are managed in one file.
That would impose a personal experiment on the whole team and not resolve the overlap.

C. **[CORRECT]** Tools from all configured servers are available at once, so differentiate the two tool descriptions (or remove the personal tool) to eliminate the overlap.
Correct. Overlapping descriptions cause misrouting when both servers' tools are present.

D. **[INCORRECT]** Only the user-scoped server is active when both exist, so the alternating behavior must come from a model defect that should be reported to the Anthropic support team.
Tools from all configured servers are discovered and available simultaneously.

Key concept: Simultaneous MCP servers and overlap (Task 2.4, 2.1).
Exam reasoning: Alternating between similar tools signals both are visible and indistinct.
Production lesson: Everything configured is in play at once. Keep tools distinct.

Q22 | Domain 2: Tool Design & MCP Integration | Scenario 4: Developer Productivity with Claude

Task statement 2.5 | Intermediate | Single answer

Correct answer: D - Grep for the message to find the file, Read it to identify the function, then Grep for that function's name to find callers.

Why the correct answer is correct: Use Grep to find entry points like error messages, Read to follow flows, then Grep again for callers, building understanding incrementally rather than reading everything.

Option-by-option:

A. **[INCORRECT]** Glob for files with 'quota' in their names, then Read all of them in full to find the message and check who calls the function.
Filenames may not mention quota and reading many files wastes context.

B. **[INCORRECT]** Use Bash to concatenate every source file into one output and scan for the message.
This floods the context and bypasses the targeted search tools.

C. **[INCORRECT]** Read the project's README and infer which module prints the message from its description.
Documentation may be outdated or silent on specific strings.

D. **[CORRECT]** Grep for the message to find the file, Read it to identify the function, then Grep for that function's name to find callers.

Correct. Each step narrows the search and uses the content-search tool appropriately.

Key concept: Incremental tracing (Task 2.5).
Exam reasoning: A literal string is best found by content search, and callers by another content search.
Production lesson: Search, read, search again.

Q23 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.1 | Intermediate | Single answer

Correct answer: D - Split it into focused files under .claude/rules/ such as testing.md and deployment.md, scoping topic files with paths frontmatter where they apply to certain files, and keep only universal guidance in CLAUDE.md.

Why the correct answer is correct: The .claude/rules/ directory organizes topic-specific rule files as an alternative to a monolithic CLAUDE.md, improving maintainability.

Option-by-option:

A. **[INCORRECT]** Keep one file but move the least-used sections to the bottom, so that Claude is likely to ignore them and merge collisions become less frequent in the top half.
Reordering doesn't reduce what is loaded and doesn't help with merge collisions.

B. **[INCORRECT]** Move the whole file into each engineer's ~/.claude/CLAUDE.md so no one collides on merges.
User-level files aren't shared, so the team would lose consistency.

C. **[INCORRECT]** Delete everything not related to testing, since tests are the main thing that must be consistent.
This throws away valid guidance.

D. **[CORRECT]** Split it into focused files under .claude/rules/ such as testing.md and deployment.md, scoping topic files with paths frontmatter where they apply to certain files, and keep only universal guidance in CLAUDE.md.
Correct. Topic-specific files are easier to maintain and can be loaded as needed.

Key concept: Modular configuration (Task 3.1, 3.3).
Exam reasoning: Merge collisions and unneeded content are symptoms of a monolithic file.
Production lesson: Modularize shared instructions by topic.

Q24 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.2 | Intermediate | Single answer

Correct answer: A - Set context: fork in the skill's frontmatter so the brainstorming runs in an isolated context and only the outcome returns to the main session.

Why the correct answer is correct: context: fork runs skills in an isolated sub-agent context, preventing verbose or exploratory outputs from polluting the main conversation.

Option-by-option:

A. **[CORRECT]** Set context: fork in the skill's frontmatter so the brainstorming runs in an isolated context and only the outcome returns to the main session.
Correct. context: fork isolates exploratory context from the main conversation.

B. **[INCORRECT]** Put the skill in ~/.claude/skills/ so that each engineer's copy runs separately.
Moving it to user scope doesn't change where the output is added.

C. **[INCORRECT]** Add allowed-tools: Read so that the skill cannot write anything during the brainstorm.
Restricting tools affects what the skill can do, not where its output goes.

D. **[INCORRECT]** Add the brainstorming prompt to CLAUDE.md so that it runs automatically at the start of every session and keeps the options in view for the whole team.
That would clutter every session and does not isolate anything.

Key concept: Skill isolation with context: fork (Task 3.2).
Exam reasoning: The complaint is about clutter and influence on the main thread.
Production lesson: Isolate exploratory work from decision-making context.

Q25 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.3 | Intermediate | Single answer

Correct answer: D - A .claude/rules/ file with paths: ["**/*.graphql"] that holds the conventions.

Why the correct answer is correct: Glob-pattern rules apply conventions to files by type regardless of directory location, an advantage over directory-level CLAUDE.md files when files are spread across the codebase.

Option-by-option:

A. **[INCORRECT]** A long section in the root CLAUDE.md that applies to everything, relying on Claude to notice GraphQL files.
This loads irrelevant content in all sessions and relies on inference.

B. **[INCORRECT]** A skill that engineers invoke before editing a schema.
Skills are on demand, so conventions would be skipped when someone forgets.

C. **[INCORRECT]** A CLAUDE.md in each of the twelve package directories, each containing a copy of the conventions.
Twelve copies drift apart, and nested files don't cover schemas elsewhere.

D. **[CORRECT]** A .claude/rules/ file with paths: ["**/*.graphql"] that holds the conventions.
Correct. A glob rule applies to matching files regardless of directory.

Key concept: Glob-scoped rules for scattered files (Task 3.3).
Exam reasoning: The files are scattered but share a type, so a path glob is the natural key.
Production lesson: Scope by file pattern when the pattern, not the directory, defines the convention.

Q26 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.4 | Challenging | Single answer

Correct answer: A - Use plan mode to explore the codebase and evaluate both approaches' implications before choosing and implementing one.

Why the correct answer is correct: Plan mode suits tasks with architectural implications and choices between integration approaches with different infrastructure requirements.

Option-by-option:

A. **[CORRECT]** Use plan mode to explore the codebase and evaluate both approaches' implications before choosing and implementing one.
Correct. Multiple valid approaches with different infrastructure needs call for planning.

B. **[INCORRECT]** Start with direct execution of the simpler streaming approach and switch to a queue only if problems appear in production.
Deferring an infrastructure decision until production risks costly rework.

C. **[INCORRECT]** Ask for the final code immediately with a detailed prompt describing the streaming design.
This presumes the decision has been made before the trade-offs were explored.

D. **[INCORRECT]** Implement both designs in parallel with direct execution and keep whichever works first.
Building both wastes effort and still lacks an explicit evaluation.

Key concept: Plan mode for architectural choices (Task 3.4).
Exam reasoning: Two designs with different infrastructure is the textbook trigger for planning.
Production lesson: Decide the architecture before you pay for the code.

Q27 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.5 | Challenging | Single answer, secondary: D4

Correct answer: B - Provide two or three concrete input/output examples, including one that shows the nullable-to-Optional mapping.

Why the correct answer is correct: Concrete input/output examples are the most effective way to communicate expected transformations when prose descriptions are interpreted inconsistently; 2-3 examples including edge cases are typical.

Option-by-option:

A. **[INCORRECT]** Generate the mappers as usual and have a reviewer rename fields after the fact.
Manual repair doesn't fix the generation process.

B. **[CORRECT]** Provide two or three concrete input/output examples, including one that shows the nullable-to-Optional mapping.
Correct. Examples remove ambiguity and cover the edge case explicitly.

C. **[INCORRECT]** Ask Claude to explain in its own words what the naming convention means before generating.
A paraphrase doesn't ensure consistent application.

D. **[INCORRECT]** Add twenty-five pages of naming guidelines to CLAUDE.md and ask Claude to read them before generating.
Long prose is still interpreted inconsistently and consumes context.

Key concept: Examples for transformation clarity (Task 3.5, 4.2).
Exam reasoning: Prose has already been tried and results vary.
Production lesson: Show the transformation, including the awkward case.

Q28 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.2 | Foundational | Single answer

Correct answer: D - In ~/.claude/commands/ in their home directory.

Why the correct answer is correct: Project-scoped commands live in .claude/commands/ (shared via version control); user-scoped commands live in ~/.claude/commands/ (personal).

Option-by-option:

A. **[INCORRECT]** In .mcp.json as a server entry named standup.
MCP configuration defines servers, not slash commands.

B. **[INCORRECT]** In the project's CLAUDE.md so it applies to their sessions only.
CLAUDE.md in the project is shared, and it doesn't define commands.

C. **[INCORRECT]** In .claude/commands/ in the project repository.
Project-scoped commands are shared with everyone who clones the repository.

D. **[CORRECT]** In ~/.claude/commands/ in their home directory.
Correct. User-scoped commands are personal and not shared via version control.

Key concept: Command scope (Task 3.2).
Exam reasoning: Personal and unshared means user scope.
Production lesson: Scope follows audience.

Q29 | Domain 3: Claude Code Configuration & Workflows | Scenario 4: Developer Productivity with Claude

Task statement 3.3 | Intermediate | Single answer

Correct answer: C - Add YAML frontmatter with paths: ["**/*.py"] so the rule loads only when working on matching files.

Why the correct answer is correct: Path-specific rules with YAML frontmatter paths load only when editing matching files, reducing irrelevant context and token usage.

Option-by-option:

A. **[INCORRECT]** Delete the rule and place its content in the user-level ~/.claude/CLAUDE.md of each developer.
That removes sharing and still loads the content everywhere.

B. **[INCORRECT]** Rename the file to python.md so Claude infers from the name that it applies only to Python.
The name does not scope loading. Frontmatter does.

C. **[CORRECT]** Add YAML frontmatter with paths: ["**/*.py"] so the rule loads only when working on matching files.
Correct. Path scoping makes the rule conditional.

D. **[INCORRECT]** Wrap the rules in a sentence saying 'ignore these unless the file is Python'.
The text still loads and consumes tokens, and relies on the model to ignore it.

Key concept: Fixing unscoped rules (Task 3.3).
Exam reasoning: Loading in every session is exactly what a missing paths field causes.
Production lesson: Unscoped rules are always-on rules.

Q30 | Domain 4: Prompt Engineering & Structured Output | Scenario 4: Developer Productivity with Claude

Task statement 4.2 | Intermediate | Single answer

Correct answer: D - Include two or three of the existing handlers as examples of the desired structure, validation, and error handling.

Why the correct answer is correct: Few-shot examples are the most effective technique for consistently formatted output when detailed instructions alone produce inconsistent results.

Option-by-option:

A. **[INCORRECT]** Ask Claude to review each generated handler against the team's written conventions and fix deviations.
Self-review against prose rules leaves the same ambiguity in place.

B. **[INCORRECT]** Generate each handler five times and pick the one that most resembles the team's pattern.
Sampling is costly and subjective.

C. **[INCORRECT]** Describe the desired pattern in a detailed paragraph, including naming and error rules, and ask Claude to follow it exactly.
Detailed prose has already produced inconsistent results.

D. **[CORRECT]** Include two or three of the existing handlers as examples of the desired structure, validation, and error handling.
Correct. Few-shot examples demonstrate the format and conventions concretely.

Key concept: Few-shot for convention adherence (Task 4.2).
Exam reasoning: Existing handlers are ready-made examples of the target.
Production lesson: Show it, don't just say it.

Q31 | Domain 5: Context Management & Reliability | Scenario 4: Developer Productivity with Claude

Task statement 5.1 | Intermediate | Single answer

Correct answer: B - Summarize the key findings from phase one and inject the summary into the initial prompt of each phase-two subagent.

Why the correct answer is correct: Summarize key findings from one exploration phase before spawning subagents for the next phase, injecting summaries into the initial context.

Option-by-option:

A. **[INCORRECT]** Tell each phase-two subagent to read the phase-one agent's conversation to find out what it learned.
Subagents don't have access to another agent's conversation.

B. **[CORRECT]** Summarize the key findings from phase one and inject the summary into the initial prompt of each phase-two subagent.
Correct. Summaries carry the needed context forward without raw exploration output.

C. **[INCORRECT]** Run phase two in the same session as phase one with no subagents, so it keeps all raw output in context.
This accumulates verbose discovery output and risks context degradation.

D. **[INCORRECT]** Skip phase one so the subagents' first task is to map everything they need themselves.
That wastes effort and repeats exploration across subagents.

Key concept: Phase-to-phase context passing (Task 5.4, 5.1).
Exam reasoning: Repeated mapping shows the later agents lack the earlier findings.
Production lesson: Pass conclusions forward, not transcripts.

Q32 | Domain 5: Context Management & Reliability | Scenario 4: Developer Productivity with Claude

Task statement 5.4 | Intermediate | Single answer

Correct answer: C - Have the agent maintain a scratchpad file of key findings (classes, flows, decisions) and consult it at the start of each day and for subsequent questions.

Why the correct answer is correct: Scratchpad files that record key findings persist across context boundaries and counteract context degradation in extended exploration.

Option-by-option:

A. **[INCORRECT]** Disable file writes so that the agent concentrates on keeping the facts in its working memory.
Working memory in context is exactly what degrades over long sessions.

B. **[INCORRECT]** Open a new session each day and let it start the analysis from scratch to avoid bias.
That repeats work and loses prior findings.

C. **[CORRECT]** Have the agent maintain a scratchpad file of key findings (classes, flows, decisions) and consult it at the start of each day and for subsequent questions.
Correct. A persistent scratchpad preserves findings across context boundaries.

D. **[INCORRECT]** Ask the agent to try harder to remember what it learned yesterday and to keep its answers consistent with the conclusions it reached then, whatever they were.
Context doesn't persist across sessions by willpower.

Key concept: Scratchpad files (Task 5.4).
Exam reasoning: Repeated exploration and contradictions show findings are not being retained.
Production lesson: Write down what you learn.

Q33 | Domain 1: Agentic Architecture & Orchestration | Scenario 5: Claude Code for Continuous Integration

Task statement 1.6 | Intermediate | Single answer, secondary: D4

Correct answer: D - Run the per-file passes in parallel, then run the integration pass afterward using their outputs.

Why the correct answer is correct: Per-file local analysis passes are independent and can run in parallel. The integration pass depends on their outputs, so it follows.

Option-by-option:

A. **[INCORRECT]** Run the per-file passes and the integration pass all in parallel and merge their outputs at the end.
The integration pass would lack the local findings it is meant to combine.

B. **[INCORRECT]** Run the integration pass first so that the per-file passes already know the cross-file concerns.
The integration pass needs the per-file findings as input, so it cannot run first.

C. **[INCORRECT]** Run everything strictly one after another in file order so each pass sees the previous file's comments.
Sequential local passes add time and bias later files toward earlier comments.

D. **[CORRECT]** Run the per-file passes in parallel, then run the integration pass afterward using their outputs.
Correct. Local passes are independent, and the integration pass depends on their results.

Key concept: Sequencing in multi-pass review (Task 1.6, 4.6).
Exam reasoning: Dependencies, not convenience, determine which steps can overlap.
Production lesson: Parallelize independent work and sequence dependent work.

Q34 | Domain 1: Agentic Architecture & Orchestration | Scenario 5: Claude Code for Continuous Integration

Task statement 1.6 | Challenging | Single answer

Correct answer: A - Add an initial classification step that identifies the type of change and routes only the relevant passes, defaulting to the full review when the type is unclear.

Why the correct answer is correct: Decomposition should match the workflow: coordinators and pipelines should select which stages to run based on the task's characteristics rather than always routing through the full pipeline.

Option-by-option:

A. **[CORRECT]** Add an initial classification step that identifies the type of change and routes only the relevant passes, defaulting to the full review when the type is unclear.
Correct. Adaptive routing avoids needless work while keeping a safe fallback.

B. **[INCORRECT]** Skip review entirely for any PR whose title contains the word 'docs' or 'fixtures', since those titles almost always describe harmless changes to non-code files.
Titles are unreliable, and skipping review risks missing real code changes.

C. **[INCORRECT]** Run the full pipeline anyway but suppress all findings below a certain severity on small PRs.
Work and cost remain, and real issues could be suppressed.

D. **[INCORRECT]** Ask developers to label every PR with its type and run only the passes for that label.
Manual labels are inconsistent and shift effort to developers.

Key concept: Adaptive routing in a pipeline (Task 1.6, 1.2).
Exam reasoning: A sizeable share of PRs does not need the full set of passes.
Production lesson: Match the amount of process to the task, with a safe default.

Q35 | Domain 1: Agentic Architecture & Orchestration | Scenario 5: Claude Code for Continuous Integration

Task statement 1.6 | Intermediate | Select 2, secondary: D4

Correct answer: D, E - Split the review into focused per-file passes for local issues plus a separate integration pass for cross-file data flow. / Apply the same explicit review criteria and output format to every per-file pass so depth and standards are uniform.

Why the correct answer is correct: Split large reviews into per-file local passes and an integration pass to avoid attention dilution; explicit shared criteria help consistent classification.

Option-by-option:

A. **[INCORRECT]** Switch to a model with a larger context window so that all 20 files can be given the same attention in one pass.
A larger window doesn't fix attention quality across many files.

B. **[INCORRECT]** Ask developers to resubmit the PR in batches of three or four files before review.
This shifts work to developers without improving the system.

C. **[INCORRECT]** Accept only findings that appear in at least two of three full-PR review runs.
Consensus filtering can suppress real bugs found inconsistently and doesn't fix depth.

D. **[CORRECT]** Split the review into focused per-file passes for local issues plus a separate integration pass for cross-file data flow.
Correct. This avoids attention dilution and handles cross-file concerns explicitly.

E. **[CORRECT]** Apply the same explicit review criteria and output format to every per-file pass so depth and standards are uniform.
Correct. Shared criteria produce consistent judgments across files.

Key concept: Multi-pass review and consistency (Task 1.6, 4.1, 4.6).
Exam reasoning: The symptoms are uneven depth and contradictions across files, which map to the two correct moves.
Production lesson: Make each pass small and make each pass use the same rubric.

Q36 | Domain 2: Tool Design & MCP Integration | Scenario 5: Claude Code for Continuous Integration

Task statement 2.2 | Challenging | Single answer, secondary: D5

Correct answer: B - Retry locally with bounded backoff, and propagate to the coordinator only if it still fails, including what was attempted and any partial results.

Why the correct answer is correct: Subagents should implement local recovery for transient failures and propagate to the coordinator only errors they cannot resolve, along with partial results and what was attempted.

Option-by-option:

A. **[INCORRECT]** Abort the entire PR review and fail the CI job so that nothing is posted without full coverage.
One transient failure shouldn't invalidate the other file reviews.

B. **[CORRECT]** Retry locally with bounded backoff, and propagate to the coordinator only if it still fails, including what was attempted and any partial results.
Correct. Local recovery first, informative propagation second.

C. **[INCORRECT]** Propagate every timeout to the coordinator immediately, which then decides whether to retry the call or to give up on that file's review altogether.
This makes the coordinator handle routine transient errors that the subagent could resolve.

D. **[INCORRECT]** Ignore the timeout and continue the review of that file without search results, without noting the gap.
Unreported missing evidence produces misleading review conclusions.

Key concept: Local recovery and propagation (Task 2.2, 5.3).
Exam reasoning: A transient timeout is the classic case for a local retry.
Production lesson: Handle what you can locally and report what you can't.

Q37 | Domain 2: Tool Design & MCP Integration | Scenario 5: Claude Code for Continuous Integration

Task statement 2.4 | Challenging | Single answer, secondary: D3

Correct answer: B - Define the server in the project's .mcp.json using environment variable expansion for the token, and supply the token to the CI job as a secret.

Why the correct answer is correct: Project-scoped .mcp.json with environment variable expansion supports shared tooling and credentials without committing secrets. User-scoped ~/.claude.json is personal.

Option-by-option:

A. **[INCORRECT]** Skip MCP in CI and ask Claude to guess issue details from branch names.
Guessing produces unreliable results where real data is available.

B. **[CORRECT]** Define the server in the project's .mcp.json using environment variable expansion for the token, and supply the token to the CI job as a secret.
Correct. The CI runner has no personal user configuration, but it reads project config and environment.

C. **[INCORRECT]** Hard-code the Jira token in .mcp.json so that CI and local use the same literal values.
Committing a literal token exposes the credential.

D. **[INCORRECT]** Copy the developer's ~/.claude.json into the repository so that CI can read it from there, and remove the personal entries that are not needed.
That file may contain unrelated personal settings and secrets that should not be committed.

Key concept: Project versus user MCP scope in CI (Task 2.4, 3.6).
Exam reasoning: A fresh CI runner has no developer's home-directory configuration.
Production lesson: Anything CI needs must come from the repository and the environment.

Q38 | Domain 3: Claude Code Configuration & Workflows | Scenario 5: Claude Code for Continuous Integration

Task statement 3.6 | Intermediate | Single answer

Correct answer: C - Retry a bounded number of times with backoff; if it still fails, mark the advisory check skipped with a clear message rather than blocking the merge.

Why the correct answer is correct: CI runs should be reliable and observable: retry transient failures a bounded number of times, report outcomes clearly, and match the failure policy to whether a check is blocking or advisory.

Option-by-option:

A. **[INCORRECT]** Remove the error handling so that failures are ignored silently and never appear in logs.
Silent failures make the system unobservable.

B. **[INCORRECT]** Fail the pipeline immediately on any error from the review step, so that engineers always know the review ran and nobody can merge code that skipped it.
Blocking merges because an advisory check had a temporary outage is disproportionate.

C. **[CORRECT]** Retry a bounded number of times with backoff; if it still fails, mark the advisory check skipped with a clear message rather than blocking the merge.
Correct. Bounded retries address transient errors and the advisory nature avoids blocking merges.

D. **[INCORRECT]** Retry indefinitely until the step succeeds so the review is always posted.
Unbounded retries can hang the pipeline.

Key concept: CI failure handling (Task 3.6).
Exam reasoning: Advisory status and a temporary cause point to bounded retries without blocking.
Production lesson: Tie failure behavior to how critical the check is.

Q39 | Domain 3: Claude Code Configuration & Workflows | Scenario 5: Claude Code for Continuous Integration

Task statement 3.5 | Intermediate | Single answer

Correct answer: B - Address them one at a time in sequence, since the problems are independent and each fix can be verified separately.

Why the correct answer is correct: Provide all issues in a single message when they interact; fix them sequentially when they are independent.

Option-by-option:

A. **[INCORRECT]** Ask Claude to ignore two of them and fix only the first, since the others may disappear.
Unrelated failures will not disappear.

B. **[CORRECT]** Address them one at a time in sequence, since the problems are independent and each fix can be verified separately.
Correct. Independent problems can be resolved sequentially.

C. **[INCORRECT]** Disable the lint, type, and snapshot checks until all modules are stable.
Disabling checks removes the verification.

D. **[INCORRECT]** Combine all three into one message so Claude sees the whole CI report at once and can plan the fixes together.
Combining helps when problems interact, but these are independent, and separate fixes are easier to verify.

Key concept: Sequential versus combined feedback (Task 3.5).
Exam reasoning: The stem states the causes are unrelated.
Production lesson: Batch only what is entangled.

Q40 | Domain 3: Claude Code Configuration & Workflows | Scenario 5: Claude Code for Continuous Integration

Task statement 3.6 | Intermediate | Single answer

Correct answer: A - The diff or its commit hash, the prompt and CLAUDE.md versions used, the model and settings, and the raw structured output.

Why the correct answer is correct: Reproducibility requires logging inputs (diff, prompt, project context), configuration (model, settings), and structured outputs.

Option-by-option:

A. **[CORRECT]** The diff or its commit hash, the prompt and CLAUDE.md versions used, the model and settings, and the raw structured output.
Correct. These allow reproducing and explaining the result.

B. **[INCORRECT]** Only the final comment text, since that is the artifact that mattered to developers.
The comment alone cannot show how it was produced.

C. **[INCORRECT]** A screenshot of the pull request page at the time the comment was posted.
A screenshot has no inputs or settings.

D. **[INCORRECT]** The names of the developers who reacted to the comment.
Reactions say nothing about the generation process.

Key concept: Logging for reproducibility (Task 3.6).
Exam reasoning: Reproduction requires inputs and configuration, not just outputs.
Production lesson: If you can't replay it, you can't audit it.

Q41 | Domain 3: Claude Code Configuration & Workflows | Scenario 5: Claude Code for Continuous Integration

Task statement 3.5 | Foundational | Single answer

Correct answer: A - Provide specific test cases with example inputs and expected outputs for the null and edge cases, and ask Claude to make them pass.

Why the correct answer is correct: Provide specific test cases with example input and expected output to fix edge case handling, such as null values.

Option-by-option:

A. **[CORRECT]** Provide specific test cases with example inputs and expected outputs for the null and edge cases, and ask Claude to make them pass.
Correct. Concrete examples fix edge case handling.

B. **[INCORRECT]** Ask Claude to restate the requirement and proceed only when it repeats it correctly.
Restating does not ensure correct implementation.

C. **[INCORRECT]** Remove null handling from scope and document that nulls are unsupported.
That avoids the requirement rather than meeting it.

D. **[INCORRECT]** Explain the same behavior a third time with greater emphasis and add exclamation marks to the key requirements, so the expectations are unmistakable.
Emphasis doesn't make the requirement clearer.

Key concept: Examples and tests for edge cases (Task 3.5).
Exam reasoning: Two prose attempts have failed, so switch to concrete cases.
Production lesson: Edge cases are best specified as examples.

Q42 | Domain 3: Claude Code Configuration & Workflows | Scenario 5: Claude Code for Continuous Integration

Task statement 3.6 | Intermediate | Single answer, secondary: D4

Correct answer: D - Block merges on the deterministic checks, keep Claude's findings advisory with human approval, and tighten only after measuring precision.

Why the correct answer is correct: Deterministic checks provide dependable gates; model findings with false positives are better advisory until measured precision supports stricter use, with human approval retained.

Option-by-option:

A. **[INCORRECT]** Make all checks advisory so developers are never delayed by automation.
Failing tests and type errors should block.

B. **[INCORRECT]** Make Claude's review the only blocking check, since it understands the code better than static tools do and can catch issues no linter would find.
False positives would block valid merges.

C. **[INCORRECT]** Block on any Claude finding of any severity to be safe.
Blocking on unvalidated judgments erodes trust in the pipeline.

D. **[CORRECT]** Block merges on the deterministic checks, keep Claude's findings advisory with human approval, and tighten only after measuring precision.
Correct. Deterministic results are reliable gates, and model judgments start advisory.

Key concept: Blocking versus advisory checks (Task 3.6, 4.1).
Exam reasoning: A mix of deterministic and judgment-based signals should be gated differently.
Production lesson: Earn blocking status with measured accuracy.

Q43 | Domain 4: Prompt Engineering & Structured Output | Scenario 5: Claude Code for Continuous Integration

Task statement 4.1 | Foundational | Single answer

Correct answer: A - List the specific categories to report, such as SQL built from user input, hard-coded secrets, and missing authorization checks, and state which minor items to skip.

Why the correct answer is correct: Explicit criteria that define which issues to report and which to skip outperform vague or confidence-based instructions for reducing false positives.

Option-by-option:

A. **[CORRECT]** List the specific categories to report, such as SQL built from user input, hard-coded secrets, and missing authorization checks, and state which minor items to skip.
Correct. Specific categorical criteria define what to flag and what to ignore.

B. **[INCORRECT]** Add 'only report issues you are highly confident about, and prefer silence over speculation' to the existing prompt so the output is cleaner and more focused for every PR.
General confidence instructions don't improve precision the way specific criteria do.

C. **[INCORRECT]** Remove the security instruction and run a general code review instead.
This dilutes focus and doesn't address precision.

D. **[INCORRECT]** Add 'be thorough but not noisy' to the prompt so that the model balances the two.
This is another vague instruction with no definition.

Key concept: Explicit review criteria (Task 4.1).
Exam reasoning: Speculative remarks come from an undefined target.
Production lesson: Define the target in concrete categories.

Q44 | Domain 4: Prompt Engineering & Structured Output | Scenario 5: Claude Code for Continuous Integration

Task statement 4.5 | Challenging | Single answer

Correct answer: D - Refine the prompt on a representative sample of a few hundred documents first, then submit the full batch once first-pass success is high.

Why the correct answer is correct: Use prompt refinement on a sample set before batch-processing large volumes to maximize first-pass success and reduce resubmission costs.

Option-by-option:

A. **[INCORRECT]** Move the full run to the synchronous API so failures are discovered one at a time.
That forgoes batch savings and doesn't improve the prompt.

B. **[INCORRECT]** Split the 40,000 documents into 400 batches of 100 and submit them all at once.
Splitting doesn't change the failure rate.

C. **[INCORRECT]** Submit the full batch now and fix any failures through resubmission afterward, because batch pricing is cheap and the failures can be handled in bulk.
A 30% failure rate means large rework, even at discounted pricing.

D. **[CORRECT]** Refine the prompt on a representative sample of a few hundred documents first, then submit the full batch once first-pass success is high.
Correct. Sample-based refinement avoids paying for large-scale resubmissions.

Key concept: Pre-batch prompt refinement (Task 4.5).
Exam reasoning: The known 30% failure rate would be multiplied at scale.
Production lesson: Fix the prompt where it's cheap to iterate.

Q45 | Domain 4: Prompt Engineering & Structured Output | Scenario 5: Claude Code for Continuous Integration

Task statement 4.1 | Intermediate | Single answer

Correct answer: C - Define each severity level with explicit criteria and a concrete code example illustrating it.

Why the correct answer is correct: Define explicit severity criteria with concrete code examples for each level to achieve consistent classification.

Option-by-option:

A. **[INCORRECT]** Ask Claude to label severity using its best judgment and be consistent with previous reviews.
Each run has no memory of earlier labels.

B. **[INCORRECT]** Remove severity from the output and let developers decide how important each finding is.
This loses prioritization information.

C. **[CORRECT]** Define each severity level with explicit criteria and a concrete code example illustrating it.
Correct. Explicit levels with examples produce consistent classification.

D. **[INCORRECT]** Report the severity as a number from 1 to 100 so small differences average out.
A finer scale adds noise without defining meaning.

Key concept: Severity criteria (Task 4.1).
Exam reasoning: Identical patterns receiving different labels indicates undefined levels.
Production lesson: Anchor each severity with an example.

Q46 | Domain 4: Prompt Engineering & Structured Output | Scenario 5: Claude Code for Continuous Integration

Task statement 4.4 | Intermediate | Single answer

Correct answer: D - Add a detected_pattern field to each finding naming the code construct that triggered it, so dismissals can be analyzed by pattern.

Why the correct answer is correct: Adding detected_pattern fields to structured findings enables analysis of false positive patterns when developers dismiss findings.

Option-by-option:

A. **[INCORRECT]** Add a free-form 'comments' field where the model explains its general impressions of the PR and its overall quality in its own words each time.
Unstructured commentary can't be aggregated.

B. **[INCORRECT]** Add a timestamp field so you can see at which hour findings are dismissed most.
Time of day doesn't indicate which constructs cause false positives.

C. **[INCORRECT]** Add a boolean field helpful that the model sets itself on each finding.
The model's self-assessment doesn't reflect developer judgment.

D. **[CORRECT]** Add a detected_pattern field to each finding naming the code construct that triggered it, so dismissals can be analyzed by pattern.
Correct. Pattern tags let you find which triggers produce false positives.

Key concept: Feedback loop design (Task 4.4).
Exam reasoning: The question is which constructs produce dismissed findings.
Production lesson: Instrument outputs so you can learn from them.

Q47 | Domain 4: Prompt Engineering & Structured Output | Scenario 5: Claude Code for Continuous Integration

Task statement 4.3 | Intermediate | Single answer, secondary: D2

Correct answer: B - 'any', which guarantees that some tool is called while letting Claude choose the right one.

Why the correct answer is correct: tool_choice 'any' guarantees the model calls a tool but lets it choose which; 'auto' allows text; forced selection names one tool.

Option-by-option:

A. **[INCORRECT]** No tool_choice setting but a stricter system prompt reminding Claude to always call a tool.
A prompt reminder is not a guarantee.

B. **[CORRECT]** 'any', which guarantees that some tool is called while letting Claude choose the right one.
Correct. 'any' guarantees structured output when several schemas exist and the type is unknown.

C. **[INCORRECT]** A forced selection of report_findings for every PR.
That would force the wrong schema for test plans and documentation changes.

D. **[INCORRECT]** 'auto', which lets Claude decide whether to call a tool or reply in prose.
This is the current behavior that allows prose replies.

Key concept: tool_choice options (Task 4.3, 2.3).
Exam reasoning: Unknown type plus a requirement for structured output rules out both 'auto' and forcing one tool.
Production lesson: Pick the weakest setting that gives the guarantee you need.

Q48 | Domain 1: Agentic Architecture & Orchestration | Scenario 6: Structured Data Extraction

Task statement 1.5 | Foundational | Single answer

Correct answer: B - A PostToolUse hook that parses each tool result into a decimal amount and ISO currency code before the model sees it.

Why the correct answer is correct: PostToolUse hooks normalize heterogeneous data formats from different tools before the model processes them.

Option-by-option:

A. **[INCORRECT]** A request to the parser vendors to standardize on one format so the problem goes away.
That depends on third parties and does not fix the pipeline today.

B. **[CORRECT]** A PostToolUse hook that parses each tool result into a decimal amount and ISO currency code before the model sees it.
Correct. Normalization happens deterministically before the model reasons.

C. **[INCORRECT]** A system prompt paragraph explaining all three formats and asking the model to convert them carefully.
The model still performs the conversion each time, and mistakes can occur.

D. **[INCORRECT]** A few-shot example in the extraction prompt that shows how to read the European format.
Examples help but don't guarantee correct handling of every variant.

Key concept: PostToolUse normalization (Task 1.5).
Exam reasoning: Mixed formats from different tools are the situation hooks are built for.
Production lesson: Normalize at the boundary.

Q49 | Domain 1: Agentic Architecture & Orchestration | Scenario 6: Structured Data Extraction

Task statement 1.5 | Intermediate | Single answer

Correct answer: C - A pre-call hook or tool-level check on create_record that enforces uniqueness atomically on the invoice's natural key (for example a unique constraint) and redirects duplicates.

Why the correct answer is correct: Hooks that intercept outgoing tool calls enforce compliance rules deterministically. Duplicate prevention based on a natural key is a hard rule suited to code.

Option-by-option:

A. **[INCORRECT]** A system prompt instruction for each worker to avoid processing invoices that look as if they were already handled by another worker earlier in the run.
Workers cannot see each other's progress, so the instruction cannot be followed reliably.

B. **[INCORRECT]** A nightly job that deletes duplicate records after they have been created.
Cleanup afterward leaves a window of bad data and may propagate duplicates downstream.

C. **[CORRECT]** A pre-call hook or tool-level check on create_record that enforces uniqueness atomically on the invoice's natural key (for example a unique constraint) and redirects duplicates.
Correct. An atomic, deterministic check on the outgoing call prevents duplicates even when workers run in parallel.

D. **[INCORRECT]** A requirement that the model explain in its response why it believes the invoice is new.
An explanation is not a check and may be wrong.

Key concept: Pre-call enforcement and duplicate prevention (Task 1.5, 1.4).
Exam reasoning: Parallel workers lack shared awareness, so the check must live in shared code.
Production lesson: Enforce uniqueness where the write happens.

Q50 | Domain 1: Agentic Architecture & Orchestration | Scenario 6: Structured Data Extraction

Task statement 1.5 | Challenging | Single answer, secondary: D5

Correct answer: B - Keep the hook, but store the original raw tool output with a reference ID alongside the normalized values so each field can be traced to its source.

Why the correct answer is correct: Normalization hooks should preserve provenance: retain raw outputs and references so extracted values remain traceable to their source.

Option-by-option:

A. **[INCORRECT]** Have the model write the original text into its response alongside every normalized field.
That inflates output, and model-copied text can itself be altered.

B. **[CORRECT]** Keep the hook, but store the original raw tool output with a reference ID alongside the normalized values so each field can be traced to its source.
Correct. The model sees clean data while provenance remains available.

C. **[INCORRECT]** Keep only the normalized values and tell the auditor that normalization is deterministic.
Determinism doesn't let anyone verify against the original.

D. **[INCORRECT]** Remove the hook and let the model see the raw OCR text again, so there is only a single version of the data and nothing to reconcile for the auditor later.
This brings back the format errors the hook solved.

Key concept: Provenance with normalization (Task 1.5, 5.6).
Exam reasoning: The requirement is traceability while keeping the benefit of consistent inputs.
Production lesson: Transform for the model, but never lose the original.

Q51 | Domain 2: Tool Design & MCP Integration | Scenario 6: Structured Data Extraction

Task statement 2.1 | Foundational | Single answer

Correct answer: C - 'Extracts header fields (vendor, invoice number, date, currency) and line items from a vendor invoice or credit note PDF. Do not use for point-of-sale receipts (use extract_receipt) or contracts.'

Why the correct answer is correct: Tool descriptions are the primary mechanism for selection and should include input formats, outputs, edge cases, and boundaries relative to similar tools.

Option-by-option:

A. **[INCORRECT]** 'Extracts structured data from documents such as invoices, receipts, statements, and other financial paperwork submitted by users, and returns the results as clean JSON that is ready for downstream use.'
This overlaps with the other extraction tools and gives no boundary.

B. **[INCORRECT]** 'Used for invoices. See the documentation page for more details about supported file types and options.'
A pointer to external documentation is not visible to the model during selection.

C. **[CORRECT]** 'Extracts header fields (vendor, invoice number, date, currency) and line items from a vendor invoice or credit note PDF. Do not use for point-of-sale receipts (use extract_receipt) or contracts.'
Correct. It states inputs, outputs, an edge case, and boundaries against similar tools.

D. **[INCORRECT]** 'A powerful and flexible tool for extracting all kinds of information from business documents quickly and accurately.'
Marketing language says nothing about inputs, outputs, or when to use it.

Key concept: Writing differentiating descriptions (Task 2.1).
Exam reasoning: Only one description tells the model what is in scope and out of scope.
Production lesson: Descriptions should explain what the tool is for and what it is not for.

Q52 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.5 | Foundational | Single answer

Correct answer: D - A unique custom_id on each request, used to correlate each result to its source document.

Why the correct answer is correct: The Message Batches API uses custom_id fields to correlate request and response pairs.

Option-by-option:

A. **[INCORRECT]** The timestamp at which each result was returned, sorted ascending.
Completion time doesn't identify which request produced it.

B. **[INCORRECT]** The length of each extracted text, used to find the matching document.
Text lengths are not unique, so many different documents would match the same value.

C. **[INCORRECT]** Relying on the order in which requests were submitted, since the service returns results in that order.
Batch results are not guaranteed to be ordered like the requests.

D. **[CORRECT]** A unique custom_id on each request, used to correlate each result to its source document.
Correct. custom_id is the supported way to match results to requests.

Key concept: custom_id correlation (Task 4.5).
Exam reasoning: Position-based joining is fragile when results may arrive in a different order.
Production lesson: Always carry an explicit key.

Q53 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.3 | Intermediate | Single answer

Correct answer: B - A nullable date field plus prompt normalization rules (convert to ISO 8601, return null if no date appears) alongside the strict schema.

Why the correct answer is correct: Include format normalization rules in prompts alongside strict output schemas, and make fields nullable when sources may lack the information.

Option-by-option:

A. **[INCORRECT]** A schema with no date field, since dates are unreliable in documents.
Removing the date field drops useful data that downstream systems need whenever a document does state it.

B. **[CORRECT]** A nullable date field plus prompt normalization rules (convert to ISO 8601, return null if no date appears) alongside the strict schema.
Correct. Normalization rules handle formatting and nullability prevents fabrication.

C. **[INCORRECT]** A required date field and a prompt telling the model to infer a plausible date from context whenever none is present in the document, so that the field is never empty.

Inferring a date invents data.

D. **[INCORRECT]** A free-text date field so the model can copy exactly what the document says.
Free text pushes normalization to downstream consumers.

Key concept: Normalization rules and nullable fields (Task 4.3).
Exam reasoning: Inconsistent formats and missing values each need their own handling.
Production lesson: Combine schema constraints with prompt-level rules.

Q54 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.4 | Intermediate | Select 2

Correct answer: C, E - A line item missing its quantity although the quantity is plainly printed in the document. / A total returned as the string '1,234.50' that fails a numeric type check.

Why the correct answer is correct: Retries work for format and structural errors where the information is present, and fail when the information is absent from the source.

Option-by-option:

A. **[INCORRECT]** A vendor tax ID the schema requires, but which does not appear anywhere in the document.
Retrying cannot produce information that is absent and may cause fabrication.

B. **[INCORRECT]** A scanned page that is too blurry for any reader to make out the amount.
Illegible source content can't be recovered by feedback.

C. **[CORRECT]** A line item missing its quantity although the quantity is plainly printed in the document.
Correct. The data exists in the source, so feedback can steer the model to include it.

D. **[INCORRECT]** A purchase order number that exists only in a separate document not provided to the model.
The information is external to the provided source.

E. **[CORRECT]** A total returned as the string '1,234.50' that fails a numeric type check.
Correct. A format mismatch can be corrected given the error.

Key concept: Resolvable versus unresolvable errors (Task 4.4).
Exam reasoning: Each correct case has the data in the source; each wrong case doesn't.
Production lesson: Retry what is recoverable, route the rest.

Q55 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.5 | Intermediate | Single answer

Correct answer: A - Synchronous API for the urgent invoices, and the Message Batches API for the archive.

Why the correct answer is correct: Match the API approach to latency requirements: synchronous for blocking or urgent workflows, batch for latency-tolerant bulk processing.

Option-by-option:

A. **[CORRECT]** Synchronous API for the urgent invoices, and the Message Batches API for the archive.
Correct. Match the API to each workload's latency tolerance.

B. **[INCORRECT]** Synchronous API for both, because consistent behavior is easier to reason about.
It gives up large savings on the archive that can easily tolerate delay.

C. **[INCORRECT]** Message Batches API for both, with a polling loop for the urgent invoices to check progress.
Batch processing has no latency guarantee, which is unsuitable for urgent work.

D. **[INCORRECT]** Message Batches API for the archive, and for urgent invoices submit one-item batches.
One-item batches still carry the batch processing window.

Key concept: Choosing sync versus batch (Task 4.5).
Exam reasoning: One workload is time-sensitive and one is bulk.
Production lesson: Different latency needs justify different paths.

Q56 | Domain 4: Prompt Engineering & Structured Output | Scenario 6: Structured Data Extraction

Task statement 4.5 | Foundational | Single answer

Correct answer: A - Submit the batch with a custom_id per request, poll for completion (results may take up to 24 hours), then retrieve results and correlate by custom_id.

Why the correct answer is correct: Batches are asynchronous with up to a 24-hour window, no latency guarantee, polling for completion, and custom_id correlation.

Option-by-option:

A. **[CORRECT]** Submit the batch with a custom_id per request, poll for completion (results may take up to 24 hours), then retrieve results and correlate by custom_id.

Correct. This matches how batches work.

B. **[INCORRECT]** Submit the batch and expect results within 30 seconds, with a guaranteed latency SLA.
No latency SLA is guaranteed.

C. **[INCORRECT]** Submit the batch and receive all results immediately in the same response, since batches are synchronous, which then need to be correlated with the requests by their order.
Batches are asynchronous and complete later.

D. **[INCORRECT]** Submit each request one at a time and block until each completes before sending the next.
That is synchronous use, not batch processing.

Key concept: Batch workflow basics (Task 4.5).
Exam reasoning: The key facts are asynchrony, polling, and custom_id.
Production lesson: Know the shape of the workflow before designing around it.

Q57 | Domain 5: Context Management & Reliability | Scenario 6: Structured Data Extraction

Task statement 5.5 | Intermediate | Select 2

Correct answer: C, D - Have the model output field-level confidence and set review thresholds by calibrating against a labeled validation set. / Route extractions whose sources are ambiguous or contradictory to human review first.

Why the correct answer is correct: Route extractions with low model confidence or ambiguous/contradictory source documents to human review; calibrate field-level confidence on labeled validation sets.

Option-by-option:

A. **[INCORRECT]** Review the 200 documents with the longest text, since longer documents have more errors.
Length is a weak proxy and untested.

B. **[INCORRECT]** Review a fixed 4% chosen alphabetically by vendor name.
Alphabetical selection has no relationship to risk.

C. **[CORRECT]** Have the model output field-level confidence and set review thresholds by calibrating against a labeled validation set.
Correct. Calibrated field-level confidence directs review to the riskiest fields.

D. **[CORRECT]** Route extractions whose sources are ambiguous or contradictory to human review first.
Correct. Ambiguous or conflicting sources are the highest-risk cases.

E. **[INCORRECT]** Trust the model's overall self-rated confidence for each document, uncalibrated, and review the lowest 200.
Uncalibrated self-reported confidence is an unreliable ranking.

Key concept: Review routing (Task 5.5).
Exam reasoning: Capacity is limited, so the routing signal needs to be meaningful.
Production lesson: Spend scarce human attention where error risk is highest.

Q58 | Domain 5: Context Management & Reliability | Scenario 6: Structured Data Extraction

Task statement 5.6 | Challenging | Single answer, secondary: D4

Correct answer: A - Verify programmatically that each quote is an exact substring of the source text at the stated page, and flag mismatches for review or retry.

Why the correct answer is correct: Source-grounded extraction needs verifiable provenance; code can check that supporting quotes are exact substrings, distinguishing validation from trust in the model's claims.

Option-by-option:

A. **[CORRECT]** Verify programmatically that each quote is an exact substring of the source text at the stated page, and flag mismatches for review or retry.
Correct. A deterministic check confirms the quote is real.

B. **[INCORRECT]** Accept paraphrases as long as the meaning matches the source.
Paraphrases cannot be located in the document and reduce verifiability.

C. **[INCORRECT]** Ask the model to confirm in a follow-up message that each of its quotes is accurate and appears verbatim in the source document at the page it cited.
A model reaffirming its own output is not independent verification.

D. **[INCORRECT]** Drop the quote field because paraphrased quotes cause confusion.
Losing the quote loses traceability.

Key concept: Provenance verification (Task 5.6, 4.4).
Exam reasoning: The failure is quotes that do not exist in the source, which string matching detects.
Production lesson: Verify provenance mechanically wherever possible.

Q59 | Domain 5: Context Management & Reliability | Scenario 6: Structured Data Extraction

Task statement 5.6 | Intermediate | Single answer

Correct answer: D - The document date or collection date for each value, so the later amendment is recognized as a revision rather than a conflict.

Why the correct answer is correct: Require publication or collection dates in structured outputs so temporal differences are not misinterpreted as contradictions.

Option-by-option:

A. **[INCORRECT]** A boolean marking the larger value as correct.
Larger isn't necessarily correct and the flag discards context.

B. **[INCORRECT]** A note saying the values are 'approximately the same'.
A vague note doesn't help interpret the difference.

C. **[INCORRECT]** Only the most recently ingested value, with the earlier one deleted.
Deleting the earlier value loses history.

D. **[CORRECT]** The document date or collection date for each value, so the later amendment is recognized as a revision rather than a conflict.
Correct. Dates let the reader distinguish revision from contradiction.

Key concept: Temporal data handling (Task 5.6).
Exam reasoning: The two values come from different dates.
Production lesson: Dates turn contradictions into timelines.

Q60 | Domain 5: Context Management & Reliability | Scenario 6: Structured Data Extraction

Task statement 5.5 | Challenging | Single answer

Correct answer: D - Use a labeled validation set to measure accuracy at different confidence levels, per document type and field, and pick the threshold meeting the target within reviewer capacity.

Why the correct answer is correct: Field-level confidence scores should be calibrated using labeled validation sets, and accuracy validated by document type and field before automating.

Option-by-option:

A. **[INCORRECT]** Pick the threshold that sends exactly as many documents to review as reviewers can handle in a day, and accept whatever accuracy results in the auto-accepted set at that level.
Capacity fit doesn't ensure the accepted fields are accurate.

B. **[INCORRECT]** Ask the model what threshold it believes is appropriate and use its answer.
The model has no access to your accuracy data.

C. **[INCORRECT]** Choose 0.9 because it is a common default and works well in many systems.
A default has no relation to your data's calibration.

D. **[CORRECT]** Use a labeled validation set to measure accuracy at different confidence levels, per document type and field, and pick the threshold meeting the target within reviewer capacity.
Correct. The threshold is based on measured accuracy and respects capacity.

Key concept: Threshold calibration (Task 5.5).
Exam reasoning: A threshold must be derived from measured performance.
Production lesson: Calibrate with data, not defaults.

---

7. Domain-level performance review guide

Use your tracker. A domain below roughly 70% on a paper deserves a focused review before the next paper. Look at the task statements and concepts listed for each domain and re-read the matching sections of the official Exam Guide.

Domain 1 - Agentic Architecture & Orchestration (27% official weight; 16 questions here)

Task	Topic	Questions
1.1	Agentic loops for autonomous task execution	1, 5
1.2	Coordinator-subagent orchestration	2, 6, 17, 19
1.3	Subagent invocation, context passing, spawning	3, 4, 16
1.5	Agent SDK hooks: interception and normalization	48, 49, 50
1.6	Task decomposition strategies	33, 34, 35
1.7	Session state, resumption, and forking	18

Lessons to carry forward: Handle every documented stop_reason deliberately. An 'else' branch that means 'done' is a bug waiting to happen. Put cross-agent concerns in the agent that has cross-agent visibility. Constrain subagents by configuration, not by hoping they behave. Treat metadata as first-class fields in every handoff. Let the model route within the loop. Put hard rules in hooks. Check the plan before you blame the workers.

Domain 2 - Tool Design & MCP Integration (18% official weight; 11 questions here)

Task	Topic	Questions
2.1	Tool interfaces: descriptions and boundaries	9, 51
2.2	Structured error responses for MCP tools	10, 36
2.3	Tool distribution and tool_choice	8, 11
2.4	MCP server integration (Claude Code and agents)	7, 21, 37
2.5	Built-in tools: Read, Write, Edit, Bash, Grep, Glob	20, 22

Lessons to carry forward: Resources describe what exists; tools do things. Least privilege improves safety and tool-selection reliability. One tool, one job, one contract. Use the error category to choose the recovery, not a reflex. Constrain at the tool, not in the prompt. Prefer the smallest tool that does the job.

Domain 3 - Claude Code Configuration & Workflows (20% official weight; 12 questions here)

Task	Topic	Questions
3.1	CLAUDE.md hierarchy, scoping, modularity	23
3.2	Custom slash commands and skills	24, 28
3.3	Path-specific rules	25, 29
3.4	Plan mode vs direct execution	26
3.5	Iterative refinement techniques	27, 39, 41
3.6	Claude Code in CI/CD pipelines	38, 40, 42

Lessons to carry forward: Modularize shared instructions by topic. Isolate exploratory work from decision-making context. Scope by file pattern when the pattern, not the directory, defines the convention. Decide the architecture before you pay for the code. Show the transformation, including the awkward case. Scope follows audience.

Domain 4 - Prompt Engineering & Structured Output (20% official weight; 12 questions here)

Task	Topic	Questions
4.1	Explicit criteria; reducing false positives	43, 45
4.2	Few-shot prompting	30
4.3	Structured output via tool use and JSON schemas	47, 53
4.4	Validation, retry, feedback loops	46, 54

Task	Topic	Questions
4.5	Batch processing (Message Batches API)	44, 52, 55, 56
4.6	Multi-instance and multi-pass review	12

Lessons to carry forward: Use confidence to prioritize, and calibrate it before relying on it. Show it, don't just say it. Define the target in concrete categories. Fix the prompt where it's cheap to iterate. Anchor each severity with an example. Instrument outputs so you can learn from them.

Domain 5 - Context Management & Reliability (15% official weight; 9 questions here)

Task	Topic	Questions
5.1	Context management across long interactions	31
5.3	Error propagation across multi-agent systems	14, 15
5.4	Context in large codebase exploration	32
5.5	Human review workflows; confidence calibration	57, 60
5.6	Provenance and uncertainty in multi-source synthesis	13, 58, 59

Lessons to carry forward: Preserve disagreement and its context. Fail informatively and degrade gracefully. Report what worked and what didn't, side by side. Pass conclusions forward, not transcripts. Write down what you learn. Spend scarce human attention where error risk is highest.

8. Final revision checklist

Tick each item when you can explain it without looking at the key. Items are drawn from the concepts tested in this paper.

Done	Concept	First seen
[ ]	Complete stop_reason handling in the agentic loop	Q1
[ ]	Coordinator responsibilities	Q2
[ ]	AgentDefinition configuration	Q3
[ ]	Structured context passing	Q4
[ ]	Model-driven versus pre-configured control flow	Q5
[ ]	Coordinator decomposition quality and refinement	Q6
[ ]	Resources for catalogs, tools for actions	Q7
[ ]	Scoped tool access	Q8
[ ]	Splitting generic tools	Q9
[ ]	Error categories and recovery	Q10
[ ]	Constrained tools	Q11
[ ]	Confidence-based routing	Q12
[ ]	Conflicting statistics and temporal data	Q13
[ ]	Error propagation strategy	Q14
[ ]	Partial results with error context	Q15
[ ]	Partitioning, explicit context, and parallel spawning	Q16
[ ]	Single versus multi-agent decisions	Q17
[ ]	Resume versus fresh start	Q18
[ ]	Coordinator iterative refinement	Q19
[ ]	Selecting the right built-in tool	Q20
[ ]	Simultaneous MCP servers and overlap	Q21
[ ]	Incremental tracing	Q22
[ ]	Modular configuration	Q23
[ ]	Skill isolation with context: fork	Q24
[ ]	Glob-scoped rules for scattered files	Q25
[ ]	Plan mode for architectural choices	Q26
[ ]	Examples for transformation clarity	Q27
[ ]	Command scope	Q28
[ ]	Fixing unscoped rules	Q29
[ ]	Few-shot for convention adherence	Q30
[ ]	Phase-to-phase context passing	Q31
[ ]	Scratchpad files	Q32
[ ]	Sequencing in multi-pass review	Q33
[ ]	Adaptive routing in a pipeline	Q34
[ ]	Multi-pass review and consistency	Q35
[ ]	Local recovery and propagation	Q36
[ ]	Project versus user MCP scope in CI	Q37
[ ]	CI failure handling	Q38
[ ]	Sequential versus combined feedback	Q39
[ ]	Logging for reproducibility	Q40

Done	Concept	First seen
[ ]	Examples and tests for edge cases	Q41
[ ]	Blocking versus advisory checks	Q42
[ ]	Explicit review criteria	Q43
[ ]	Pre-batch prompt refinement	Q44
[ ]	Severity criteria	Q45
[ ]	Feedback loop design	Q46
[ ]	tool_choice options	Q47
[ ]	PostToolUse normalization	Q48
[ ]	Pre-call enforcement and duplicate prevention	Q49
[ ]	Provenance with normalization	Q50
[ ]	Writing differentiating descriptions	Q51
[ ]	custom_id correlation	Q52
[ ]	Normalization rules and nullable fields	Q53
[ ]	Resolvable versus unresolvable errors	Q54
[ ]	Choosing sync versus batch	Q55
[ ]	Batch workflow basics	Q56
[ ]	Review routing	Q57
[ ]	Provenance verification	Q58
[ ]	Temporal data handling	Q59
[ ]	Threshold calibration	Q60

Before the next paper: classify each wrong answer as (a) knowledge gap, (b) misread of the stem or qualifier, or (c) a plausible-distractor trap. See CCAF_Final_Revision_Guide.pdf for the method.